

Переносное устройство для тестирования и мониторинга сетей

Рабочий перевод бакалаврской работы на русский язык
(оригинал: Přenosné zařízení pro testování a monitorování sítí)

Михаил Муканов
VŠB – Technická univerzita Ostrava, FEI
Руководитель: Ing. Pavel Nevlud

Редакция от 28.09.2026

Аннотация

Эта бакалаврская работа посвящена проектированию и реализации переносного устройства для тестирования и мониторинга компьютерных сетей. Устройство построено на одноплатном компьютере Raspberry Pi с сенсорным дисплеем, двумя внешними сетевыми адаптерами и аккумуляторным модулем и управляется без подключённого компьютера. Оба тестовых порта логически разделены пространствами имён ядра Linux, поэтому прибор может одним портом генерировать трафик, а другим независимо его анализировать. Устройство содержит анализатор трафика протоколов IPv4 и IPv6 со статистикой, разбором кадров и фильтрами, которые выполняются в ядре, генератор пакетов, измерение задержки, пропускной способности, потерь и джиттера инструментом iperf3, включая развёртку по размерам кадра, аудит сообщений Router Advertisement и поиск соседей IPv6. Точность анализатора проверена независимым пересчётом захваченного трафика, измерения проведены и на переключке между портами, и против компьютеров с Linux и Windows. В работе очерчены и пределы использованной платформы, прежде всего пропускная способность, ограниченная шиной USB 2.0 примерно 276 Мбит/с. В типичном режиме прибор работает от аккумулятора почти семь часов.

Ключевые слова: компьютерные сети; мониторинг сети; генерирование трафика; анализатор трафика; Raspberry Pi; network namespaces; IPv4; IPv6; пропускная способность; потери пакетов.

Оглавление

1	Введение	5
2	Теоретические основы и анализ проблематики	7
2.1	Протоколы сетевого уровня и их различия в производительности	7
2.2	Адресация и автоматическая настройка протокола IPv6	8
2.3	Генерирование сетевого трафика	9
2.4	Мониторинг и аудит безопасности сетевого трафика	11
2.5	Архитектура Linux Network Namespaces (netns)	11
2.6	Анализ проблематики и существующих решений	12
3	Проект и реализация, оценка результатов	14
3.1	Проект переносного устройства	14
3.1.1	Питание от аккумулятора	15
3.1.2	Программная архитектура	16
3.2	Реализация генератора и анализатора трафика	17
3.2.1	Пользовательский интерфейс	17
3.2.2	Анализатор трафика	18
3.2.3	Генератор пакетов	19
3.2.4	Измерение задержки, пропускной способности и потерь	20
3.2.5	Автоконфигурация адресов IPv6	22
3.2.6	Аудит сообщений Router Advertisement	23
3.2.7	Поиск соседей	23
3.3	Оценка достигнутых результатов	24
3.3.1	Проверка автоконфигурации адресов	24
3.3.2	Проверка аудита IPv6	25
3.3.3	Точность и пределы анализатора	26
3.3.4	Производительность генератора пакетов	27
3.3.5	Измерение пропускной способности	27
3.3.6	Подключение обычного компьютера	28
3.3.7	Питание и время автономной работы	30
3.3.8	Сканирование сети и проверка изоляции пространств имён	31
3.3.9	Дефекты, выявленные при повторной проверке	31
4	Заключение	33

Собственный вклад автора	33
Ограничения и рекомендации по дальнейшему развитию	34
Литература	35
А Исходные коды измерительных скриптов	37
В Обзор программного обеспечения прибора	38

Список иллюстраций

3.1	Собранный прототип с аккумуляторным модулем	15
3.2	Программная архитектура анализатора и разделение на пространства имён	16
3.3	Главный экран управляющего приложения с результатами последних тестов	18
3.4	Экран TRAFFIC при захвате трафика на порту мониторинга	19
3.5	Разбор захваченного сообщения Router Advertisement по слоям	19
3.6	Генератор пакетов: отправлено портом <code>snd0</code> , посчитано на <code>mon0</code>	20
3.7	Тест <code>ping</code> на перемычке между портами с объяснением отсутствующей цели	21
3.8	Измерение по UDP против ноутбука	21
3.9	Физическое подключение анализатора к станции в лаборатории	24
3.10	Аудит сообщения Router Advertisement, принятого от второго порта прибора	25
3.11	Поиск соседей по IPv6 и сканирование ARP	26
3.12	Измерение пропускной способности по TCP и IPv6 против ноутбука	28
3.13	Пропускная способность и кадры в секунду в зависимости от размера кадра	29
3.14	Результат развёртки по размерам кадра на дисплее прибора	29
3.15	Режим сервера: ноутбук измеряет пропускную способность против анализатора	30
3.16	Напряжение аккумулятора во время разрядного теста в типичном режиме	31
3.17	Сканирование Nmap через тестовый порт	32

Список таблиц

2.1	Значение флагов в сообщении Router Advertisement	9
3.1	Потребление прибора в разных режимах, измерено на входе 5 В	16
3.2	Флаги в сообщении Router Advertisement до и после исправления	22
3.3	Адреса, выданные станции на обоих тестовых портах	24
3.4	Поиск другой стороны на одном кабеле обоими протоколами	26
3.5	Число кадров по протоколам: анализатор прибора и независимый пересчёт сохр- нённого файла	26
3.6	Пропускная способность по TCP, тест длительностью десять секунд	27
3.7	Развёртка по размерам кадра по UDP, анализатор против ноутбука	28
В.1	Файлы программного обеспечения прибора	38

Глава 1

Введение

Компьютерные сети сегодня — базовая инфраструктура, от надёжности которой зависит работа учреждений и производственных предприятий. С ростом числа подключённых устройств и постепенным внедрением протокола IPv6 наряду с привычным IPv4 растёт и сложность задач, которые администратору сети приходится решать прямо на месте. Вопросы при этом бывают очень конкретными: действительно ли розетка в помещении соединена с распределительным шкафом, какую пропускную способность даёт линия, какие устройства находятся в сегменте и не работают ли на них службы, которых там быть не должно.

Ответ на эти вопросы требует измерения на месте. Именно здесь обычные решения упираются в свои ограничения. Коммерческие ручные тестеры эргономичны, но закрыты: набор их функций задан производителем, и добавить в них собственную методику проверки нельзя. Ноутбук с открытыми инструментами, наоборот, полностью гибок, но в шкафу или под столом с ним работать неудобно, и перед каждым измерением его нужно настраивать вручную. Если же техник попытается подключиться к одному и тому же сегменту двумя интерфейсами, чтобы одновременно слушать трафик и создавать нагрузку, он столкнётся с конфликтом в таблицах маршрутизации операционной системы, который незаметно исказит результаты измерений.

Цель этой бакалаврской работы — спроектировать и реализовать переносное устройство для тестирования и мониторинга компьютерных сетей, которое сочетает открытость программного решения с эргономикой самостоятельного прибора. Прототип построен на одноплатном компьютере Raspberry Pi, оснащён сенсорным дисплеем, собственным управляющим приложением и аккумулятором, поэтому им можно управлять без подключённого компьютера, без физической клавиатуры и без сетевого питания. Он содержит генератор и анализатор трафика, измеряет задержку, пропускную способность, потери и джиттер и позволяет картировать сегмент и проводить разведку безопасности, одинаково для протоколов IPv4 и IPv6.

Ключевой технический элемент конструкции — логическое разделение двух тестовых интерфейсов с помощью технологии Linux Network Namespaces. Это разделение гарантирует, что создаваемая нагрузка не влияет на статистику, захваченную слушающим интерфейсом, и обеспечивается самим ядром операционной системы, а не только аккуратностью оператора при настройке. Второе направление, которому уделено повышенное внимание, — автоматическая настройка адресов в протоколе IPv6, поскольку именно от неё зависит, сможет ли прибор установить связь со станцией, на которой ничего заранее не настроено.

Текст работы разделён на четыре главы. После этого введения следует глава 2, которая излагает теоретические основы: различия протоколов сетевого уровня, адресацию и автоконфигурацию протокола IPv6, генерирование сетевого трафика и измерение его параметров, инструменты мониторинга и аудита трафика и архитектуру пространств имён. Её заключительная часть сравнивает существующие решения и формулирует требования к проектируемому устройству. Глава 3 описывает проект прибора с аппаратной и программной стороны, реализацию генератора и анализатора трафика и оценку результатов, проверенных измерениями. Заключение подводит итоги, выделяет

собственный вклад автора и намечает направления дальнейшего развития прототипа.

Глава 2

Теоретические основы и анализ проблематики

Эта глава посвящена теоретическим основам, на которых построено спроектированное устройство для тестирования и мониторинга. Понимание этих понятий необходимо для правильной интерпретации данных, полученных при анализе сети.

2.1 Протоколы сетевого уровня и их различия в производительности

Современные компьютерные сети опираются на сосуществование протоколов IPv4 и IPv6 [1]. Обе версии выполняют на сетевом уровне одну и ту же задачу — адресацию станций и доставку пакета между сетями, — но различаются настолько, что измерительный прибор должен обслуживать каждую из них отдельно. Одновременная работа обоих протоколов на одном интерфейсе, называемая двойным стекком, сегодня обычна в корпоративных сетях, и переносной анализатор должен уметь проверить работоспособность каждой ветви по отдельности.

Самое заметное различие — длина адреса, выросшая с 32 до 128 бит. Изменение адресного пространства, однако, не единственное вмешательство в конструкцию протокола. Заголовок IPv4 имеет переменную длину от 20 до 60 байт, поскольку за обязательной частью могут следовать дополнительные поля, и содержит, помимо прочего, собственную контрольную сумму и три поля для фрагментации. Заголовок IPv6, напротив, фиксирован и занимает 40 байт [2]. Вся дополнительная функциональность перенесена в заголовки расширения, которые выстраиваются цепочкой за основным заголовком и на существование которых указывает поле Next Header.

У такой конструкции два следствия, которые проявляются именно при измерениях. Во-первых, маршрутизатор на пути пакета выполняет меньше работы: он не пересчитывает контрольную сумму, потому что её вообще убрали из заголовка, полагаясь на защиту канального и транспортного уровней, и не фрагментирует. Фрагментировать в IPv6 может только отправитель, который для этого должен заранее выяснить наименьший размер передаваемого блока на всём пути. С этим связано требование, чтобы каждая линия передавала пакет размером не менее 1280 байт. Во-вторых, IPv6 полностью отказался от широковещательной рассылки и заменил её групповой, что прямо отражается на том, как устройства в сегменте находят друг друга; этому посвящён подраздел 2.2.

С точки зрения измерения пропускной способности существенно, что вдвое больший основной заголовок — это накладные расходы, которые вычитаются из полезной нагрузки. При одинаковом размере кадра канального уровня на данные остаётся на двадцать байт меньше, поэтому измеренная прикладная пропускная способность у IPv6 получается чуть ниже. Как показывают сравнительные измерения Tuifaiga и соавторов [3], IPv4 в локальных сетях действительно достигает несколько большей пропускной способности и меньшей задержки, чем IPv6, одинаково в Windows и Linux.

Различие, впрочем, нужно толковать осторожно. Его относительная величина зависит от того, какими блоками передаются данные: у длинных потоков с полноразмерными кадрами это единицы процентов, а у трафика из коротких пакетов может быть заметно больше. Поэтому измеренная разница между протоколами сама по себе говорит не о качестве сети, а о свойствах протокола, и при оценке нагрузочных тестов из главы 3 её нужно отличать от реальных неисправностей.

2.2 Адресация и автоматическая настройка протокола IPv6

Различие между протоколами не ограничивается размером заголовка. Для переносного анализатора важнее другой способ, которым станция вообще получает свой адрес, поскольку от него зависит, сможет ли прибор обратиться к другой стороне без всякой предварительной договорённости с её владельцем. Поэтому этому вопросу посвящён отдельный подраздел.

Адрес IPv6 имеет длину 128 бит, его структура определена архитектурой адресации [4], а сам протокол — документом [2]. Запись делится на восемь групп по четыре шестнадцатеричные цифры, причём самый длинный непрерывный участок нулевых групп можно заменить двумя двоеточиями. Для тестового устройства важны три категории адресов. Адрес локального канала из диапазона `fe80::/10` возникает на каждом интерфейсе автоматически в момент поднятия линии и действует только в пределах одного физического сегмента. Глобальный адрес маршрутизируется во всём Интернете. Между ними стоят уникальные локальные адреса, которые для нашего устройства интереснее всего.

Уникальные локальные адреса задаются префиксом `fc00::/7`, причём восьмой бит различает способ выделения; при его установке, то есть для префикса `fd00::/8`, организация сама генерирует сорокабитный идентификатор псевдослучайным образом [5]. Такой префикс с очень высокой вероятностью уникален, не маршрутизируется в Интернете и не зависит ни от какого провайдера. Именно эти свойства соответствуют природе переносного анализатора, который работает в закрытом тестовом сегменте, не имеет постоянного подключения к Интернету и чей адресный план не должен конфликтовать с адресами сети, куда прибор временно подключён.

Вторую половину адреса образует идентификатор интерфейса длиной 64 бита. Один из способов его получения, описанный в архитектуре адресации [4], исходит из физического адреса интерфейса и называется EUI-64. Сорокавосемьбитный адрес делится на две половины, между которыми вставляются два байта со значением `fffe`, а в первом байте инвертируется седьмой бит, различающий универсальное и локальное управление. Так физический адрес `00:e0:4c:68:02:16` даёт идентификатор `02e0:4cff:fe68:0216`. Для нашего устройства это практически важно: адрес другой стороны получается детерминированно, и его можно вычислить, а не прописывать жёстко в управляющем приложении.

У идентификатора, полученного из физического адреса, есть, однако, недостаток: он выдаёт оборудование станции и позволяет следить за ней, в какую бы сеть она ни подключилась. Поэтому современные операционные системы чаще используют идентификатор, который для данной сети постоянен, но из физического адреса не выводится [6], а рядом с ним ещё и временные адреса, которые регулярно меняются [7]. Одна станция поэтому обычно несёт на одном интерфейсе несколько адресов сразу и, если так настроена, для исходящей связи предпочитает временный [7]. Для измерительного прибора отсюда следует, что другая сторона может отвечать не с того адреса, к которому к ней обратились, — в практической части это оказалось существенным.

Обнаружением соседей на общей линии в IPv6 занимается механизм Neighbor Discovery [8], который заменяет протокол ARP из IPv4 и построен поверх ICMPv6. С его помощью устройства узнают физические адреса друг друга, проверяют достижимость соседей и находят маршрутизаторы в сегменте. Для нашего устройства ключевые два его сообщения. Сообщением Router Solicitation, отправляемым на групповой адрес всех маршрутизаторов, станция после поднятия интерфейса спрашивает, есть ли в сегменте маршрутизатор. Ответом служит сообщение Router Advertisement,

которым маршрутизатор объявляет себя и вместе с тем префиксы, действующие в сегменте. Частью механизма является и обнаружение дублирующихся адресов, которым станция проверяет, что выбранный ею адрес в сегменте ещё не занят.

На сообщении Router Advertisement построена автоконфигурация адресов без сохранения состояния [9]. Станция сама составляет полный адрес из объявленного префикса и собственного идентификатора интерфейса, без какого-либо сервера, который вёл бы учёт выдачи. Воспользуется ли она этой возможностью и как, определяют три флага в сообщении, значение которых сведено в таблице 2.1.

Таблица 2.1: Значение флагов в сообщении Router Advertisement

Флаг	Название	Значение для станции
M	Managed address configuration	адрес станция запрашивает у сервера DHCPv6
O	Other configuration	прочие данные, например адреса DNS, получает от сервера DHCPv6
A	Autonomous address configuration	из этого конкретного префикса станция может составить адрес сама

Принципиально, что флаги M и O относятся ко всему сообщению, а флаг A передаётся у каждого объявленного префикса отдельно. Маршрутизатор может в одном сообщении объявить несколько префиксов и для каждого решить, может ли станция вывести из него адрес самостоятельно. Оба пути не исключают друг друга и на практике обычно комбинируются, так что у станции может быть одновременно адрес, составленный без сохранения состояния, и адрес, выданный сервером.

Альтернативой с сохранением состояния является DHCPv6 [10], перенимающий роль, знакомую по IPv4: сервер ведёт учёт выданных адресов, привязывает их к идентификатору клиента и управляет сроком аренды. Ценой этого учёта он даёт администратору полный контроль над тем, какая станция какой адрес получит, и позволяет передать и другие параметры настройки.

Разница подходов для переносного тестового устройства существенна. Станция с IPv4 без работающего клиента DHCP адрес не получит никогда, потому что другого механизма протокол не предлагает. Ядро же операционной системы с IPv6 выдаёт адрес локального канала само, а при поднятом флаге A составит из объявленного префикса и адрес более широкой области действия, причём на станции не должна работать никакая служба настройки сети. Анализатор, правильно объявляющий префикс, может поэтому установить связь и с машиной, на которой ничего не настроено и над которой у оператора прибора нет никакого контроля. Проверка этого предположения на собранном прототипе описана в подразделе 3.3.1.

2.3 Генерирование сетевого трафика

Работу с сетевым трафиком можно разделить на два принципиально разных подхода. Пассивный мониторинг только наблюдает трафик и ничего в сеть не отправляет, а активное измерение само создаёт трафик и смотрит, как с ним справится сеть. Переносной анализатор должен уметь и то, и другое, потому что они отвечают на разные вопросы: пассивное наблюдение показывает, что в сегменте происходит на самом деле, активное измерение — на что сегмент способен. Этот раздел посвящён активной стороне, то есть генерированию трафика, следующий — пассивной.

Смысл генерирования в том, что в сеть отправляют трафик с известными свойствами и на приёмной стороне выясняют, что из него и как дошло. Из сравнения отправленного и принятого определяются основные параметры сети. Пропускная способность показывает, сколько данных в секунду сеть передаст. Потери показывают, какая доля отправленных пакетов не дошла. Задержку чаще всего измеряют как время оборота, то есть время между отправкой сообщения Echo Request протокола ICMP и приходом ответа, как это делает утилита ping. Наконец, джиттер (разброс за-

держки) описывает, насколько колеблется время передачи последовательных пакетов.

Генераторы трафика по уровню, на котором они работают, делятся на две группы. Генераторы потоков создают непрерывную передачу данных между клиентом и сервером и оценивают результаты на обеих сторонах соединения. Типичный представитель — инструмент `iperf3` [11]. Выбор транспортного протокола определяет, что именно измеряется. Измерение поверх TCP даёт пропускную способность, достижимую для приложения, с учётом управления потоком и восстановления после потерь, поэтому результат отражает поведение реальной передачи данных. Измерение поверх UDP, наоборот, отправляет с заранее заданной скоростью без оглядки на состояние сети и кроме пропускной способности говорит о потерях и джиттере. Различие между передачей с установлением соединения и без него на высоких скоростях подробно разбирают Gál и соавторы [12]; их работа служит методической основой для измерений в этой работе. Для содержательного результата тест должен длиться достаточно долго, чтобы управление потоком успело установиться, и измерение нужно повторять.

Вторую группу составляют генераторы пакетов, которые собирают отдельные кадры выбранного типа слой за слоем. Они служат не столько для измерения пропускной способности, сколько для проверки, пройдёт ли определённый вид трафика через сегмент и как на него отреагируют остальные устройства. В языке Python для этого есть библиотека `Scapy`, которая позволяет сложить кадр из отдельных заголовков, отправить его и обработать ответы [13]. В коммерческих тестерах для той же цели стоит специализированная аппаратура, способная отправлять точно рассчитанные по времени кадры на полной скорости линии; программный генератор на обычном компьютере ограничен производительностью процессора и точностью таймеров операционной системы.

Скорость генерируемого трафика можно выразить двумя способами — в битах в секунду и в кадрах в секунду, — и эти величины не взаимозаменяемы. Обработка каждого кадра требует и в сетевом устройстве, и в компьютере определённой постоянной работы независимо от его размера. Поэтому на коротких кадрах устройство обычно исчерпывает свою производительность раньше, чем заполнит пропускную способность линии, и его предел задаётся числом кадров в секунду; на длинных кадрах, наоборот, раньше упирается в пропускную способность линии или шины. Методика измерения производительности сетевых устройств [14] поэтому предписывает повторять измерение для ряда размеров кадра, для Ethernet это 64, 128, 256, 512, 1024, 1280 и 1518 байт, и пропускной способностью считает наибольшую скорость, при которой не теряется ни один кадр. Для IPv6 рекомендуются те же размеры кадра [15], но с оговоркой, что заголовок IPv6 на двадцать байт длиннее и самые короткие кадры могут не вместить все заголовки и данные измерительного инструмента.

Джиттер для UDP обычно оценивают способом, заимствованным из протокола RTP [16]. Для каждой пары последовательно принятых пакетов определяют разность D между разностью времён их прихода и разностью времён их отправки. Из этих разностей непрерывно вычисляется сглаженная оценка

$$J_i = J_{i-1} + \frac{|D_{i-1,i}| - J_{i-1}}{16}. \quad (2.1)$$

Поскольку используются только разности времён, часы отправителя и получателя не обязаны быть синхронизированы. По той же формуле считает джиттер и `iperf3` [11]. Для толкования результатов полезно также различать среднюю и мгновенную скорость: программный генератор может выдерживать заданную среднюю скорость и при этом отправлять короткими пачками на полной скорости линии, которые принимающее устройство с небольшим буфером может не успеть обработать.

Активное измерение нагружает сеть, поэтому важно, где оно проводится. Методы [14] разрабатывались для измерения отдельных устройств в изолированной лабораторной среде. Дополняющий документ [17] прямо предостерегает от их применения в рабочей сети: тестовый трафик может перегрузить ресурсы, общие с пользователями, а результаты к тому же будут неверными, потому что потери от чужого трафика не отличить от потерь измеряемого устройства. Переносной прибор поэтому должен ограничивать нагрузочные тесты закрытым тестовым сегментом.

2.4 Мониторинг и аудит безопасности сетевого трафика

Пассивный захват в системах на базе Linux построен на библиотеке `libpcap` и механизме фильтрации в ядре, называемом BPF [18]. Программа передаёт ядру фильтр, записанный выражением, например `tcp` или `host 10.0.1.20`, и ядро по нему отбрасывает неинтересные кадры ещё до того, как скопирует их в память приложения. Это заметно снижает нагрузку на процессор, что для одноплатного компьютера существенно. Чтобы интерфейс принимал и кадры, адресованные не ему, его нужно перевести в неразборчивый (*promiscuous*) режим. Поверх этого слоя работает `tcpdump`, который выводит захваченный трафик или сохраняет его в файл формата `pcap` для последующего разбора. Главное ограничение пассивного прослушивания — топология современных сетей: коммутатор доставляет кадры только на тот порт, где находится адрес назначения, поэтому анализатор захватит либо трафик, адресованный ему самому, либо трафик на зеркальном порту, либо на линии, в которую он включён напрямую. Это нужно помнить при толковании измеренной статистики.

Из захваченного трафика составляются сводные статистики. Анализатор Wireshark предлагает, в частности, разбивку трафика по протоколам, обзор общающихся станций с объёмом отправленных ими данных и распределение размеров пакетов [19]. Для быстрой диагностики в поле полезнее всего число кадров и бит в секунду, средний размер кадра, доли отдельных протоколов и станции с наибольшим объёмом трафика: они отвечают на вопросы, общается ли в сегменте что-нибудь, какой трафик преобладает и кто его создаёт.

У самого захвата тоже есть пределы. Если программа не успевает обрабатывать кадры так быстро, как они приходят, заполняется буфер в ядре, и дальнейшие кадры ядро выбрасывает. Статистика тогда выглядит правдоподобно, но неполна. К счастью, ядро ведёт счёт выброшенных кадров, так что надёжный анализатор должен указывать это число при каждом измерении. Предел анализатора, как и у генератора, определяется скорее числом кадров в секунду, чем объёмом данных.

Для картирования сегмента снова служит Scapy: анализатор рассылает запросы ARP на все адреса выбранного диапазона и по ответам составляет список активных станций с их физическими адресами. Как отмечает Carter [13], именно автоматизация таких задач на Python существенно расширяет возможности сетевого аудита, потому что позволяет подстроить собственные методики под конкретную среду.

Ещё одну группу образуют инструменты активной разведки безопасности, среди которых особое место занимает сканер Nmap. Его работа идёт в несколько этапов. Сначала он выясняет, какие адреса выбранного диапазона отвечают, затем у найденных станций исследует состояние выбранных портов и, наконец, по виду ответов пытается определить, какие службы на них слушают и какая операционная система установлена. Последняя задача, называемая снятием отпечатка стека, основана на мелких различиях в том, как разные реализации строят ответы на необычные комбинации флагов.

С активной разведкой неразрывно связан вопрос правомерности. Сканировать чужую сеть без согласия её администратора недопустимо, независимо от того, причиняет ли это вред. Проектируемое устройство решает этот вопрос своей конструкцией: оно работает в закрытом тестовом сегменте, который создаёт само и единственным другим участником которого является станция, подключённая кабелем напрямую. Сканирование поэтому никогда не выходит за пределы пространства, очерченного оператором, — это наряду с точностью измерений вторая причина логического разделения, описанного в следующем разделе.

2.5 Архитектура Linux Network Namespaces (netns)

Для автономного тестирования на одной физической машине используется технология Linux Network Namespaces (netns). Она обеспечивает логическую изоляцию сетевых ресурсов прямо на

уровне ядра ОС. Каждое пространство имён содержит собственные независимые экземпляры сетевого стека: изолированные таблицы маршрутизации, правила межсетевого экрана и отдельные сетевые интерфейсы.

Только что созданное пространство имён полностью пусто и содержит лишь интерфейс обратной петли, к тому же выключенный. Работоспособная сетевая среда появляется только после переноса в него физического интерфейса и его поднятия. Перенесённый интерфейс одновременно исчезает из исходного пространства, так что основная система больше о нём не знает и ничего через него отправить не может.

Изоляция касается всего сетевого стека, а не только интерфейсов. У каждого пространства свои таблицы маршрутизации, своя таблица соседей, свой набор правил пакетного фильтра и свои слушающие сокеты. Два процесса в разных пространствах могут занять один и тот же порт, не мешая друг другу, что удобно для измерительного прибора с несколькими экземплярами одной службы. Команды направляются в нужное пространство вспомогательной утилитой, которая запускает процесс прямо в нём; весь механизм входит в стандартное ядро и не требует никаких надстроек.

Если пространства имён должны, наоборот, общаться, их соединяют парой виртуальных интерфейсов типа `veth`, которая ведёт себя как воображаемый кабель между ними. В нашем анализаторе нужно ровно противоположное, поэтому такого соединения нет. Следствие: между двумя тестовыми портами нет пути, и трафик одного не попадёт в другой даже по ошибке.

Ту же задачу можно было бы решить иначе. Можно взять два отдельных компьютера, но это противоречит требованию переносимости и повышает цену. Виртуализация добавила бы накладные расходы, которые одноплатный компьютер несёт с трудом. Остаётся маршрутизация по адресу источника или привязка приложения к конкретному интерфейсу; эти решения работают, но изоляция у них — вопрос правильной настройки, и одна ошибка или инструмент, не поддерживающий привязку к интерфейсу, её разрушит. Пространства имён, напротив, обеспечивают разделение на уровне ядра, так что неправильно направленному трафику некуда уйти, и ошибка проявляется сразу и однозначно, а не тихим искажением измеренных значений.

2.6 Анализ проблематики и существующих решений

Задачу проверить состояние сетевой розетки, измерить достижимую пропускную способность или выяснить, какие устройства есть в сегменте, на практике решают тремя разными путями. У каждого свои достоинства, и именно из их сравнения вытекает задание этой работы.

Самый распространённый путь — универсальный компьютер с открытыми инструментами. Захват трафика обеспечит `tcpdump` или Wireshark, нагрузочные тесты — `iperf3`, разведку сегмента — `Nmap`. Это решение даёт наибольшую производительность и свободу, потому что измерительной станцией служит полноценный компьютер. Его слабость — эксплуатация. Ноутбук в шкафу или под столом использовать неудобно, для подключения к тестируемой сети ему обычно нужен переходник, и перед каждым измерением приходится вручную задавать адреса и проверять, куда на самом деле пойдёт трафик. Если же техник подключится к одному сегменту двумя интерфейсами, чтобы одновременно слушать и нагружать, он столкнётся с конфликтом таблиц маршрутизации, описанным в разделе 2.5: операционная система отправит трафик тем интерфейсом, который соответствует маршруту по умолчанию, а не тем, который имел в виду техник.

Второй путь — коммерческие ручные тестеры, предназначенные именно для работы в поле. Они эргономичны, сразу после включения готовы к работе и не требуют знания командной строки. Плата за удобство — закрытость. Набор функций задан производителем, добавить в прибор собственный скрипт или новую методику нельзя, а расширение обычно означает покупку старшей модели. В учебной и лабораторной среде, где важно видеть и то, как измерение проходит на самом деле, закрытая конструкция скорее мешает.

Третий путь, которому посвящена и эта работа, — одноплатный компьютер с открытой опера-

ционной системой. Пригодность платформы Raspberry Pi для этой цели подтверждена в литературе. Coşar и Kiran [20] исследовали именно производительность и измеряли поведение Raspberry Pi при анализе сетевого трафика, включая загрузку процессора и памяти. Tripathi и Kumar [21] на той же платформе одновременно запускали систему обнаружения вторжений, ловушку для атакующих и анализатор пакетов, показав, что мощности платы хватает на несколько сетевых служб сразу. Hariawan и Sunaringtyas [22] построили аналогичное решение на более новом поколении платы и применили его в домашней сети.

У этих работ общая черта. На первый план в них выходит пассивная сторона задачи — захват, обнаружение и оценка трафика, который возникает в сети независимо от измерительного устройства. Активное создание нагрузки и особенно его отделение от измерительного пути в них центральной роли не играют, как и вопрос, как управлять прибором без подключённого компьютера.

Именно здесь открывается место для задания этой работы. Цель — соединить достоинства обоих крайних подходов: сохранить открытость и расширяемость компьютера с Linux и при этом достичь эргономики ручного тестера. Из сравнения существующих решений вытекают такие требования к проектируемому устройству. Прибор должен позволять одновременно пассивно слушать и активно генерировать трафик, причём обе задачи должны вестись отдельно, чтобы нагрузка не искажала захваченную статистику. Это разделение должно обеспечиваться логически, а не аккуратностью оператора, — для этого служат пространства имён. Управление должно быть возможно прямо на устройстве, без ноутбука и без физической клавиатуры, и прибор должен на время измерений работать от собственного источника энергии. IPv6 должен поддерживаться так же глубоко, как IPv4, включая автоконфигурацию адресов на другой стороне, поскольку именно она решает, сможет ли прибор обратиться к станции, на которой ничего не подготовлено. И наконец, прибор после подключения к чужой сети не должен сам ничего отправлять; активные тесты должны выполняться только по команде оператора, а нагрузочные — только в закрытом тестовом сегменте, как требует [17].

Выполнению этих требований посвящена следующая глава.

Глава 3

Проект и реализация, оценка результатов

Эта глава описывает практическую часть работы в том порядке, в каком создавался прототип: от физического проекта прибора и его питания через программную архитектуру и реализацию генератора и анализатора трафика до оценки результатов, проверенных измерениями. Все приведённые значения измерены на собранном прототипе; там, где измерение повторялось в разных схемах подключения, схема всегда указана, потому что от неё результат существенно зависит.

3.1 Проект переносного устройства

В качестве вычислительного блока выбран одноплатный компьютер Raspberry Pi 3 Model B+. На плате стоит четырёхъядерный процессор Cortex-A53 с частотой 1,4 ГГц, 1 ГБ оперативной памяти, четыре порта USB 2.0 и беспроводной интерфейс для диапазонов 2,4 и 5 ГГц [23]. Платформа нативно поддерживает операционные системы на базе Linux, что необходимо для работы с пространствами имён и сетевыми инструментами, а её цена соответствует замыслу создать доступную альтернативу коммерческим тестерам.

Выбор платформы несёт ограничение, которое нужно знать заранее, потому что оно прямо ограничивает ценность измерений пропускной способности. Встроенный Ethernet хоть и гигабитный, но подключён к процессору через шину USB 2.0, и производитель указывает для него наибольшую достижимую пропускную способность 300 Мбит/с [23]. На той же шине сидят все порты USB, а значит, и внешние адаптеры, описанные далее. Поэтому прибор не может проверить, передаст ли линия полную гигабитную скорость; его задача — подтвердить работоспособность соединения, измерить пропускную способность в пределах платформы и сравнить протоколы в одинаковых условиях.

Отображение и управление обеспечивает пятидюймовый сенсорный дисплей с разрешением 800 на 480 точек. Изображение передаётся по HDMI, питание дисплея и данные сенсорного слоя идут через контакты GPIO, причём разъём дисплея занимает контакты 1–26. Сенсорный слой резистивный, в системе его обслуживает драйвер ADS7846. Таким образом, для работы прибору не нужны монитор, клавиатура и мышь.

Для одновременного и взаимно разделённого прослушивания и генерирования трафика одного встроенного интерфейса недостаточно, поэтому прибор дополнен двумя внешними гигабитными адаптерами USB/LAN на контроллере RTL8153. Первый служит портом мониторинга, второй — портом генерирования трафика. В первой версии прототипа эти интерфейсы получали от ядра имена `eth1` и `eth2`, порядок которых зависит от того, в каком порядке система при старте находит устройства на шине USB. При повторных загрузках оказалось, что интерфейсы меняются именами, так что роль мониторинга доставалась то одному, то другому адаптеру. Поэтому имена теперь

назначаются по физическому адресу адаптера: порт мониторинга называется `mon0`, порт генерирования `snd0`, и так при каждой загрузке. Собранный прототип показан на рисунке 3.1.

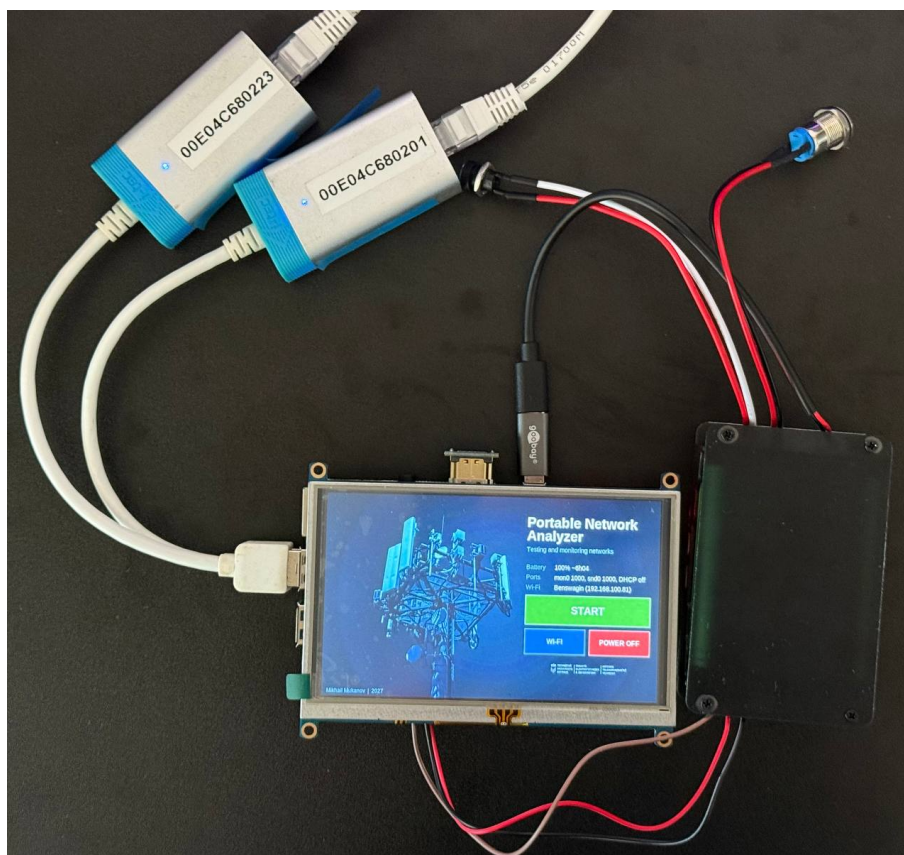


Рис. 3.1: Собранный прототип: Raspberry Pi с дисплеем, аккумуляторный модуль, кнопка питания и два подписанных адаптера USB/LAN, соединённых кабелем

3.1.1 Питание от аккумулятора

Переносной прибор должен работать без сетевого источника, поэтому он дополнен аккумуляторным модулем. Выбор модуля ограничила конструкция дисплея. Обычные платы питания для Raspberry Pi подключаются к плате снизу пружинными контактами к GPIO, но разъём дисплея занимает контакты 1–26 с обеих сторон, и плату некуда поставить. Поэтому выбран отдельный модуль *Waveshare UPS Module 3S* [24], который ставится рядом с платой. Модуль несёт три ячейки типа 18650, соединённые последовательно, преобразователь на выходное напряжение 5 В и микросхему INA219 для измерения напряжения и тока ячеек. Установлены ячейки Sony VTC6 ёмкостью 3000 мА·ч. Выход модуля питает Raspberry Pi через её собственный разъём micro-USB, так что дисплей по-прежнему питается от платы.

Ёмкость аккумулятора выбрана по заранее измеренному потреблению прибора. Потребление измерялось USB-тестером, включённым между блоком питания и Raspberry Pi, в шести режимах по десять минут; результаты сведены в таблице 3.1. Из измерений следует, что потребление определяется прежде всего объёмом передаваемых данных, а не сложностью выполняемой операции. Сканирование Nmap увеличивает ток по сравнению с простым всего на 30 мА, пассивное прослушивание на 36 мА, а нагрузочный тест *iperf3* на 366 мА.

Микросхема INA219 подключена к шине I2C. Поскольку контакты обычной шины заняты разъёмом дисплея, шина создана программно на свободных контактах 29–31. Состояние аккумулятора системная служба читает каждые десять секунд, по напряжению определяет оставшийся заряд с

Таблица 3.1: Потребление прибора в разных режимах, измерено на входе 5 В

Режим работы	Ток [мА]	Мощность [Вт]
простой, подсветка дисплея выключена	678	3,40
простой, подсветка включена	732	3,67
пассивное прослушивание (типичный режим)	768	3,89
измерение пропускной способности iperf3	1098	5,52
сканирование Nmap	762	3,83
всё одновременно (худший случай)	1242	6,10

помощью кривой разряда, измеренной на этом приборе, и записывает результат в файл, откуда управляющее приложение показывает его в строке состояния. Если напряжение ячеек в трёх измерениях подряд падает ниже 9,6 В, служба штатно выключает прибор, чтобы не повредить ни ячейки, ни файловую систему на карте памяти. Результат разрядного теста приведён в подразделе 3.3.7.

3.1.2 Программная архитектура

Программное обеспечение прибора состоит из управляющего приложения, набора самостоятельных измерительных скриптов и нескольких системных служб. Связи между этими частями показаны на рисунке 3.2.



Рис. 3.2: Программная архитектура анализатора и разделение на пространства имён

В основе лежит разделение сетевых интерфейсов на пространства имён, описанные в разделе 2.5. Порт мониторинга принадлежит пространству analyzer_monitor с адресами 10.0.1.10/24 и fd00:1::10/64, порт генерирования — пространству analyzer_sender с адресами 10.0.2.10/24 и fd00:2::10/64. Управляющее приложение остаётся в основном пространстве, где обслуживает дисплей и беспроводной интерфейс. Между двумя пространствами нет никакого маршрута, в том числе для IPv6: единственный трафик, который попадает с одного порта на другой, — это трафик, прошедший по физическому кабелю.

Из такого устройства вытекает правило, по которому организовано всё программное обеспече-

ние. Каждая функция, которой нужно работать в пространстве одного из портов, — это отдельный скрипт, потому что команда `ip netns exec` переносит в выбранное пространство весь процесс, а управляющее приложение должно оставаться в основном. Скрипты можно запускать и самостоятельно из командной строки. Свой результат они выводят последней строкой в виде `RESULT <имя скрипта> <JSON>`, которую управляющее приложение читает и показывает. Каждый такой результат приложение также сохраняет в файл вместе со временем и часовым поясом, признаком того, были ли часы прибора синхронизированы по NTP, и контрольной суммой использованного скрипта. Благодаря этому для каждого измеренного числа можно установить, когда и какой версией программы оно получено.

Пространства имён готовит системная служба, которая запускается всякий раз, когда в системе появляется адаптер с соответствующим физическим адресом, то есть и при загрузке, и при подключении адаптера на ходу. Служба создаёт пространство, если его ещё нет, переносит в него интерфейс, назначает адреса и поднимает линию. Пространство после отключения адаптера сохраняется, так что снова подключённый адаптер возвращается в то же пространство. Вторая служба следит за изменениями состояния линий и адресов в обоих пространствах и записывает их в файл состояния, из которого управляющее приложение читает состояние портов без прав администратора.

Прибор работает в так называемом режиме прибора. После включения без рабочего стола операционной системы сразу запускается управляющее приложение, примерно через 36 секунд после старта ядра. Через десять минут без касаний экран блокируется и гаснет, что экономит 0,35 Вт; подсветка дисплея управляется только механическим переключателем, программно её не погасить.

Особое внимание уделено тому, что прибор сам отправляет в сеть. Переносной прибор может оказаться в чужой рабочей сети и не должен в ней ничего нарушить. Поэтому после подключения кабеля он ничего не отправляет: ядру запрещено самостоятельно искать маршрутизатор сообщением Router Solicitation, сервер `dnsmasq`, раздающий адреса, запускается только кнопкой, а каждый активный тест — только по команде оператора. Даже с включённым `dnsmasq` прибор не выдаёт себя за шлюз по умолчанию. Сервер выдаёт подключённой станции только адрес, без параметров шлюза и сервера DNS [25], а сообщения Router Advertisement несут нулевое время жизни маршрутизатора, что по [8] означает, что станция не включит отправителя в список маршрутизаторов по умолчанию. Причина выяснилась при подключении ноутбука с Windows: тот отдаёт проводному интерфейсу приоритет над беспроводным, и если бы анализатор объявил шлюз по умолчанию, ноутбук отправлял бы через него весь трафик в Интернет, куда прибор ничего не пересылает. Так ноутбук сохраняет подключение через Wi-Fi, а с анализатором общается только в пределах тестируемого сегмента.

3.2 Реализация генератора и анализатора трафика

3.2.1 Пользовательский интерфейс

Управляющее приложение написано на Python с библиотекой Tkinter и работает в полноэкранном режиме. Главный экран разделён на двенадцать плиток, каждая из которых открывает одну функцию прибора (рисунок 3.3). На плитке после выполнения теста показывается его последний результат, так что оператор видит состояние прибора, не открывая отдельные экраны. Серым отмечены функции, которые ещё в разработке.

Все экраны функций устроены одинаково. Вверху строка состояния со временем, состоянием аккумулятора, состоянием линии и скоростью обоих портов, состоянием сервера DHCP и подключением Wi-Fi. Под ней строка итога, которая одним словом PASS, WARN или FAIL и ключевым числом подытоживает последний тест, причём цвет итога всегда дублируется словом. Внизу ряд кнопок с выбором порта, кнопками действий и кнопкой «назад», всегда на одном и том же месте.

Раскладка подчинена резистивной сенсорной панели. Такая панель распознаёт только одно ка-

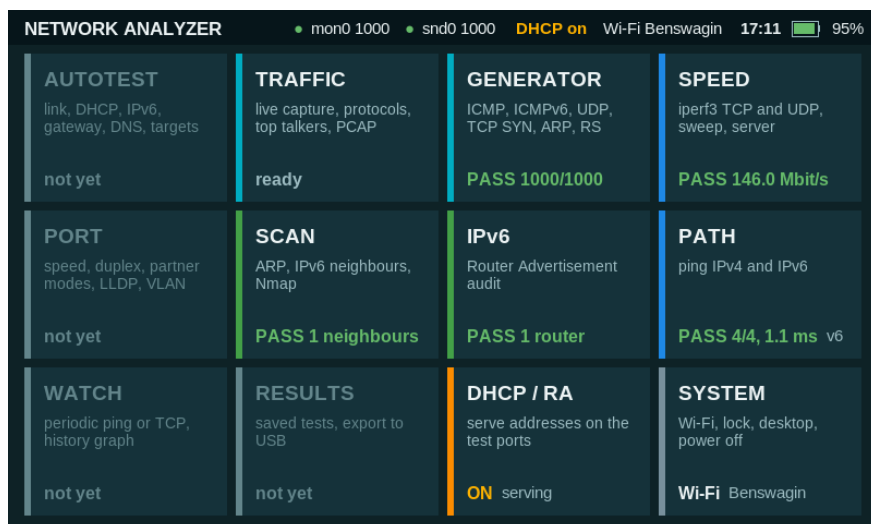


Рис. 3.3: Главный экран управляющего приложения с результатами последних тестов

сание, и точность её хуже всего у краёв рабочей области. Поэтому основные кнопки имеют размер не менее 80 на 80 точек, что на этом дисплее соответствует примерно одиннадцати миллиметрам, между соседними элементами промежутки не менее восьми точек, и ни один элемент управления не лежит ближе пятнадцати точек к краю. Соблюдение этих правил на всех экранах проверяет вспомогательный скрипт, который обходит все экраны и измеряет положение и размер каждой кнопки. Приложение не использует жесты; списки листаются кнопками. Адреса вводятся в первую очередь выбором из найденных целей, затем цифровой клавиатурой и лишь в крайнем случае, например при вводе пароля сети Wi-Fi, полной клавиатурой на экране.

3.2.2 Анализатор трафика

Пассивный анализ трафика обеспечивает скрипт `sniff_stats.py`. Сам захват выполняет `tcpdump`, который передаёт захваченные кадры скрипту в двоичном формате `pcap`. Фильтр, которым оператор ограничивает наблюдаемый трафик, передаётся `tcpdump` как выражение языка BPF и выполняется в ядре [18]; кадры, не прошедшие фильтр, до программы вообще не доходят. Скрипт разбирает у каждого кадра заголовки Ethernet, при наличии метки VLAN, IPv4 или IPv6 вместе с цепочкой заголовков расширения, а далее TCP, UDP, ICMP, ICMPv6 или ARP. Библиотека Scapy для разбора не использована, потому что на процессоре этой платы она слишком медленна для непрерывного потока кадров.

Раз в секунду скрипт выводит строку со сводными показателями: число кадров и бит в секунду, средний размер кадра, доли протоколов, соотношение трафика IPv4 и IPv6, число разных станций и пять станций с наибольшим объёмом отправленных данных. Эти показатели соответствуют статистикам анализатора Wireshark [19]. Строка содержит также несколько последних захваченных кадров с их разбором, так что управляющее приложение получает не одно сообщение на каждый кадр, а одно сводное сообщение в секунду. В конце захвата скрипт сообщает, сколько кадров ядро не успело передать и выбросило, так что возможная потеря при захвате измеряется, а не угадывается.

При проверке выяснилось, что на интерфейсах по умолчанию включены функции, которыми сетевая карта и ядро склеивают последовательные сегменты TCP в один большой пакет. Анализатор тогда видел вместо кадров размером не более 1514 байт пакеты в десятки килобайт, и счётчики и средний размер искажались. Поэтому скрипт на время захвата выключает эти функции на наблюдаемом порту и после окончания возвращает их в исходное состояние.

Поверх скрипта построен экран TRAFFIC (рисунок 3.4). В верхней части он показывает глав-

ные показатели, под ними полосу с долями протоколов, слева самые активные станции, справа текущую ленту последних кадров. Кнопка PAUSE останавливает ленту, счёт при этом продолжается, и показывает последние двести кадров постранично. Касание кадра открывает его разбор по слоям; на рисунке 3.5 показан разбор сообщения Router Advertisement с флагами и опциями. Кнопка FILTER предлагает предустановки TCP, UDP, ICMP, ARP, DHCP, ND/RA и DNS и фильтр по выбранной станции; выбранная предустановка превращается в выражение BPF, и захват перезапускается с ним. Кнопка SAVE сохраняет захваченный трафик в формате pcap для разбора в Wireshark, ход показателей в формате CSV и сводный результат в формате JSON.

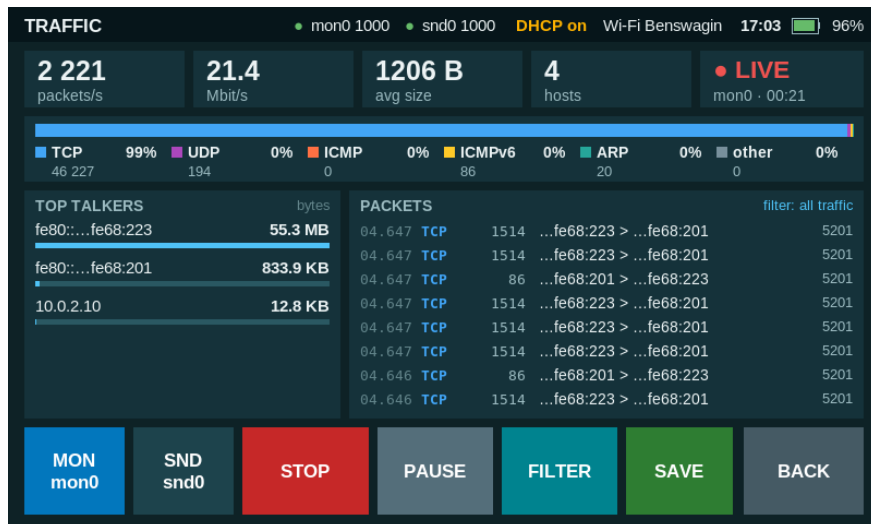


Рис. 3.4: Экран TRAFFIC при захвате трафика на порту мониторинга

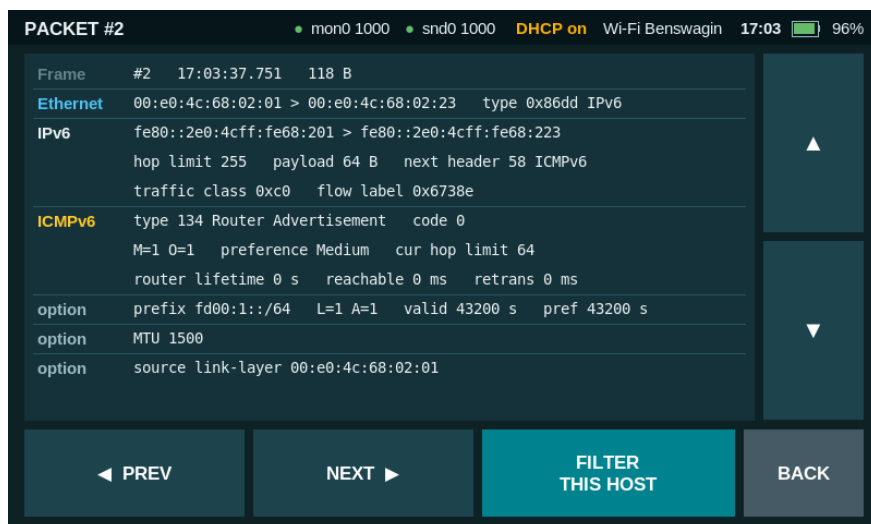


Рис. 3.5: Разбор захваченного сообщения Router Advertisement по слоям

3.2.3 Генератор пакетов

Инструмент iperf3 измеряет пропускную способность, но не позволяет отправить точно заданное число пакетов определённого типа. Для этого служит скрипт pkt_gen.py. Он умеет собирать сообщения ICMP и ICMPv6 Echo Request, датаграммы UDP, сегменты TCP с флагом SYN, запросы ARP и сообщения Router Solicitation в заданном количестве, с заданной скоростью и размером кадра. Кадры он заранее собирает библиотекой Scapy [13] и отправляет прямо на канальный уровень, так

что медленная библиотека в цикле отправки вообще не участвует и скорость ограничена только процессором и шиной. Целью могут быть только адреса тестового стенда; на другие адреса скрипт без явного подтверждения ничего не отправит.

Экран GENERATOR объединяет генератор с анализатором (рисунок 3.6). Пакеты отправляет один порт прибора, а второй порт в собственном пространстве имён считает их анализатором с фильтром по физическому адресу отправителя и выбранному типу пакета. Пакеты, таким образом, заведомо проходят по кабелю из одного пространства имён в другое, и результат сразу сравнивается: равное число отправленных и принятых пакетов даёт итог PASS, разница показывается как потеря.

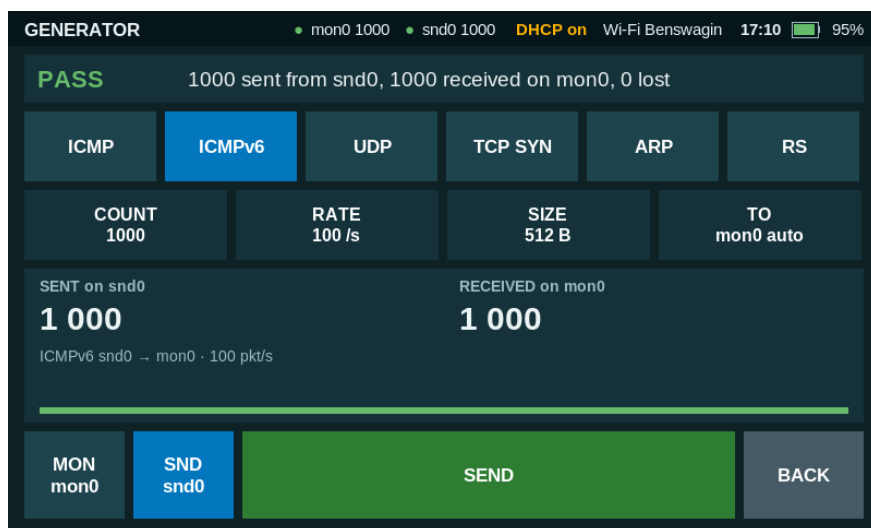


Рис. 3.6: Генератор пакетов: тысяча сообщений ICMPv6, отправленных портом `snd0` и посчитанных на порту `mon0`

3.2.4 Измерение задержки, пропускной способности и потерь

Задержку и доступность другой стороны измеряет экран PATH тестом `ping`. Пропускную способность, потери и джиттер измеряет экран SPEED инструментом `iperf3`. Обоим тестам нужна цель. В первой версии приложения адреса целей были прописаны жёстко, позже их можно было найти кнопкой, описанной в подразделе 3.2.7. Теперь, если цель не задана, тесты ищут её сами в таком порядке: адрес, который выдал подключённой станции DHCP-сервер прибора, затем соседи, известные ядру, и наконец ответ на сообщение Echo Request, отправленное на групповой адрес всех узлов. Каждый кандидат перед использованием проверяется новым запросом ARP или Neighbor Solicitation, потому что запись о выданном адресе живёт ещё часы после отключения станции. Если прибор по физическому адресу понимает, что ответил его собственный второй порт, он сообщает, что порты соединены между собой.

Если тест не находит цель или цель не отвечает, он сообщает оператору причину и рекомендацию. На перемычке между портами, например, тест по IPv4 объясняет, что порты принадлежат разным подсетям и между ними ничего не маршрутизирует, и советует использовать IPv6 (рисунок 3.7). Если станция отвечает на запрос ARP или Neighbor Solicitation, но не на `ping`, прибор сообщает, что станция подключена, но `ping` блокирует её межсетевой экран.

Нагрузочными тестами управляет скрипт `perf_test.py`, который запускает `iperf3` [11] с выводом в формате JSON и берёт числа оттуда, а не из текстового вывода. Оператор выбирает протокол TCP или UDP, версию IP, для UDP скорость отправки, длительность теста и направление, то есть кто отправляет данные — анализатор или другая сторона. Для UDP прибор кроме пропускной способности показывает число потерянных датаграмм и джиттер (рисунок 3.8). К каждому результа-

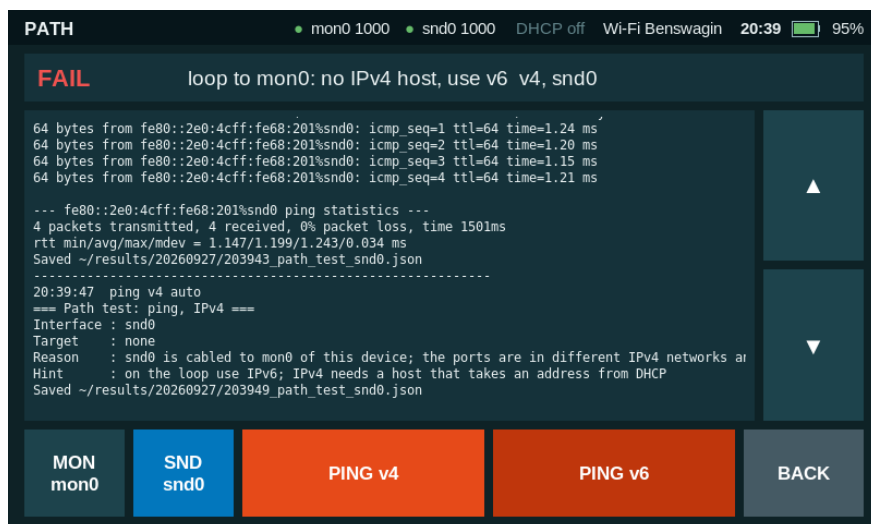


Рис. 3.7: Тест ping на перемычке между портами: цель IPv6 найдена автоматически, для IPv4 прибор объясняет, почему цели не существует

ту скрипт записывает среднюю загрузку процессора, температуру чипа в начале и в конце теста и признак того, не притормаживался ли процессор из-за температуры или питания. Измерение с притормаживанием помечается предупреждением, потому что его результат нельзя сравнивать с остальными.

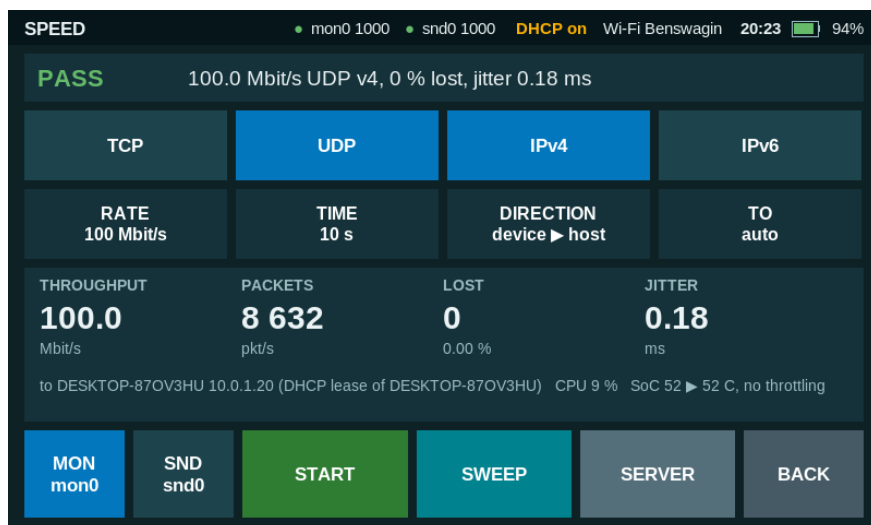


Рис. 3.8: Измерение по UDP против ноутбука: пропускная способность, датаграмм в секунду, потери и джиттер

У экрана SPEED есть ещё две функции. Кнопка SWEEP запускает измерение по UDP последовательно для семи размеров кадра, рекомендованных для Ethernet в [14], и показывает результаты таблицей. Размер кадра пересчитывается в размер данных датаграммы отдельно для IPv4 и IPv6, потому что заголовок IPv6 на двадцать байт длиннее. Кадр размером 64 байта для IPv6 собрать нельзя: заголовки Ethernet, IPv6 и UDP вместе с контрольной суммой кадра сами занимают 66 байт, а iperf3 нужно не меньше 16 байт данных. Поэтому ряд для IPv6 начинается с кадра размером 82 байта, что [15] для измерительных инструментов допускает. Кнопка SERVER, наоборот, запускает на выбранном порту сервер iperf3 и показывает на дисплее адреса, к которым должна подключиться другая сторона; измеряет тогда ноутбук против анализатора. Поскольку методы [14] не предназначены для рабочих сетей [17], измерение против адреса вне тестового стенда прибор

запускает только после повторного подтверждения, а развёртку по размерам кадра вне стенда не запускает вовсе. Если цель — второй порт того же прибора, скрипт сам запускает на нём сервер `iperf3` на время теста, так что измерение на перемычке не требует никакой подготовки.

3.2.5 Автоконфигурация адресов IPv6

Требование полной поддержки IPv6 при реализации оказалось самой трудной частью настройки — из-за различной природы двух протоколов. Если станция с IPv4 без клиента DHCP получить адрес не может, то IPv6 наряду с сервером DHCPv6 с сохранением состояния предлагает автоконфигурацию без сохранения состояния, при которой станция сама составляет адрес из префикса, объявленного в сообщении Router Advertisement. Какой путь выберет станция, определяет само сообщение: флаг M отсылает её к серверу DHCPv6, а флаг A, передаваемый у каждого префикса отдельно, только разрешает составить адрес из этого префикса [1].

Сервер `dnsmasq` работает в каждом пространстве имён отдельно и обслуживает только интерфейс своего пространства. В первой реализации диапазон адресов IPv6 был записан как один конкретный адрес: `--dhcp-range=fd00:1::20,fd00:1::20,64,12h`. Это внешне безобидное сужение диапазона привело к тому, что сервер выставил флаги в сообщении Router Advertisement так, как показано в таблице 3.2. Значения считаны на станции утилитой `rdisc6`, которая запрашивает сообщение и выводит его содержимое.

Таблица 3.2: Флаги в сообщении Router Advertisement до и после исправления

Флаг	Исходная настройка	После добавления <code>slaac</code>
M – Stateful address configuration	Yes	Yes
O – Stateful other configuration	Yes	Yes
A – Autonomous address configuration	No	Yes
On-link	Yes	Yes

Автоконфигурации мешали, таким образом, сразу два обстоятельства. Поднятый флаг M отсылал станцию к серверу DHCPv6, а опущенный флаг A у префикса запрещал составить из него адрес самостоятельно. `dnsmasq` выставляет оба именно так, если диапазон задан конкретным адресом, поскольку понимает такую запись как указание раздавать адреса только с сохранением состояния. Решением оказалось добавление ключевого слова `slaac` в тот же параметр: `--dhcp-range=fd00:1::20,fd00:1::20,slaac,64,12h`. После этой единственной правки анализатор начал объявлять префикс с поднятым флагом A, не переставая обслуживать запросы с сохранением состояния.

Эффект проявился сразу. Ядро станции составило из объявленного префикса адрес `fd00:1::2e0:4cff:fe68:21` помеченный в системном выводе признаком `proto kernel_ra`, то есть адрес, созданный самим ядром без участия какого-либо процесса в пространстве пользователя. Идентификатор интерфейса в его второй половине соответствует физическому адресу интерфейса станции `00:e0:4c:68:02:16`, преобразованному по EUI-64, так что адрес другой стороны вычисляется из её физического адреса.

Из наблюдения следует вывод важнее самого исправления настройки. Различие протоколов не в объёме ручной работы, ведь современные менеджеры сетевых подключений создают профиль сами для обоих протоколов. Различие в том, нужен ли на станции вообще какой-либо процесс: в IPv4 без клиента DHCP адрес не появится никогда, а в IPv6 ядро выдаёт адрес локального канала сразу после поднятия интерфейса и при поднятом флаге A составляет из объявленного префикса и адрес более широкой области действия.

3.2.6 Аудит сообщений Router Advertisement

Выводы предыдущего подраздела показали, что способ адресации всего сегмента определяет содержимое одного-единственного сообщения. Станция сама не проверяет, кто отправил сообщение Router Advertisement, и примет его от любого, кто пошлёт его в сегмент [8]. Для диагностического прибора отсюда следует задача уметь это сообщение захватить, разобрать и оценить, соответствует ли оно требованиям стандарта. Эту задачу выполняет скрипт `ra_audit.py`, который оператор запускает с экрана IPv6.

Скрипт сначала начинает слушать сообщения ICMPv6, а затем сам отправляет сообщение Router Solicitation на групповой адрес `ff02::2`, чтобы не ждать периодического объявления. Из захваченных сообщений он извлекает значение Cur Hop Limit, флаги M и O, предпочтение и время жизни маршрутизатора, а из опций — объявленные префиксы с их флагами и временами жизни, значение MTU, адреса серверов DNS и физический адрес отправителя. Над разобранным сообщением он выполняет набор проверок: соответствует ли Hop Limit в заголовке предписанным 255 и является ли адрес источника адресом локального канала [8], имеет ли префикс с поднятым флагом A длину 64 бита [9], не превышает ли предпочтительное время жизни время действительности, не опускается ли MTU ниже минимума 1280 байт [2] и совпадает ли физический адрес в кадре с адресом в опции. Каждое сообщение содержит ссылку на соответствующий документ прямо в тексте.

При проверке выяснилось, что делить сообщения только на свои и чужие по физическому адресу отправителя недостаточно, потому что интерфейс захвата видит и кадры, уходящие с того же порта. Прибор поэтому объявлял о присутствии маршрутизатора, когда наблюдал отправку собственного `dnsmasq`. Происхождение сообщения поэтому различается в три ступени: кадр, отправленный этим же портом; кадр со второго порта того же прибора, пойманный по кабелю; и кадр чужого маршрутизатора — единственный повод для тревоги.

Из того же механизма вытекает ограничение: прибор не может запросить сообщение Router Advertisement у сервера, работающего на этом же порту, потому что запрос уходит прямо на канальный уровень в обход сетевого стека и сервер его не замечает. Сообщение должно пройти по кабелю, поэтому аудит проверяется в схеме, где оба порта соединены одним кабелем; каждый из них тогда слышит объявления другого.

3.2.7 Поиск соседей

Жёстко прописанные адреса другой стороны годятся на лабораторном стенде, но не в поле, где у подключённого устройства может быть свой адресный план. Вывод о том, что адрес, составленный автоконфигурацией, вычисляется из физического адреса, даёт способ избавиться от этого допущения. Эту функцию обеспечивает скрипт `neigh_scan.py`, запускаемый кнопкой NEIGHBORS с экрана SCAN.

Скрипт отправляет сообщение Echo Request на групповой адрес всех узлов `ff02::1` и дополняет ответы содержимым таблицы соседей ядра. Для каждого найденного физического адреса он вычисляет идентификатор интерфейса по EUI-64 и сравнивает его с адресом локального канала, который сосед реально использует. Совпадение значит, что станция строит идентификатор классически из физического адреса; несовпадение — что она использует идентификатор, не выдающий оборудование, по [6], смысл которого — не дать следить за станцией в разных сетях. Если вычисление возможно, скрипт для каждого префикса на порту составляет адрес-кандидат и проверяет его тестом доступности, так что в результат попадает подтверждённый адрес, а не догадка. Той же кнопкой запускается и сканирование ARP в диапазоне адресов порта, так что оператор видит результат обоих протоколов рядом.

3.3 Оценка достигнутых результатов

Прототип проверялся в трёх схемах подключения. Вначале он был проверен в лаборатории компьютерных сетей, где был соединён двумя кабелями со станцией под Linux (рисунок 3.9); такая же схема с персональным компьютером под Linux позже послужила для измерения автоконфигурации. Большая часть дальнейших измерений проведена в схеме, где оба порта прибора соединены одним кабелем между собой. Такое измерение повторяемо и не требует никакого другого оборудования, но прибор в нём измеряет сам себя. Поэтому результаты дополнительно проверены с ноутбуком под Windows 11, к которому оба порта были подключены через два дополнительных адаптера USB/LAN. Эта третья схема выявила несколько обстоятельств, которые перемишка между портами скрывает; они описаны в подразделе 3.3.6.

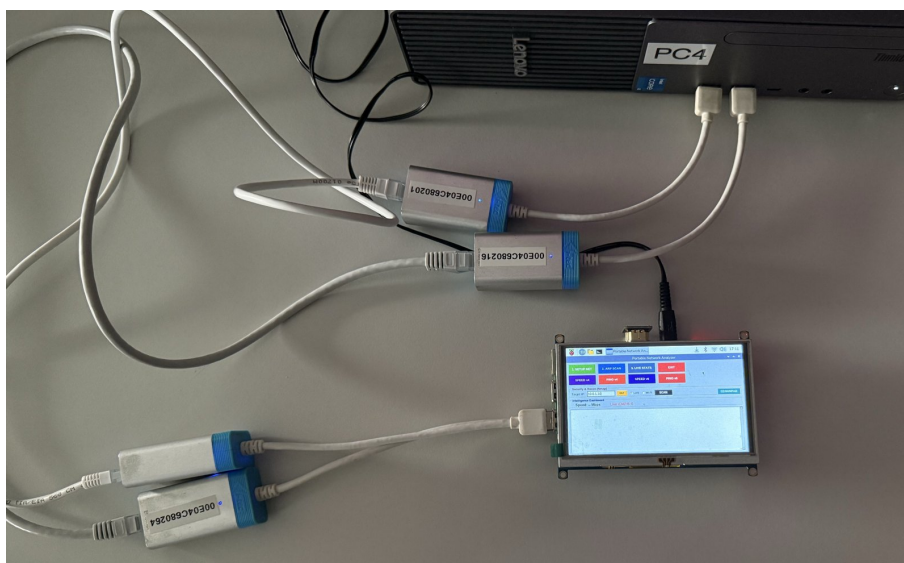


Рис. 3.9: Физическое подключение анализатора к станции в лаборатории

3.3.1 Проверка автоконфигурации адресов

Поведение автоконфигурации, описанное в подразделе 3.2.5, проверено измерением, при котором анализатор был соединён со станцией обоими портами сразу. Порт мониторинга перезапущен с добавленным ключевым словом `slaas`, а порт генерирования оставлен в исходной настройке как контрольный образец. Адреса, выданные обоим интерфейсам станции, сведены в таблице 3.3.

Таблица 3.3: Адреса, выданные станции на обоих тестовых портах

Порт анализатора	Адрес станции	Способ получения
мониторинг, A = 1	10.0.1.20	DHCPv4
мониторинг, A = 1	fd00:1::20/128	с сохранением состояния, DHCPv6
мониторинг, A = 1	fd00:1::2e0:4cff:fe68:216/64	без сохранения состояния, ядром из RA
генерирование, A = 0	10.0.2.20	DHCPv4
генерирование, A = 0	fd00:2::20/128	с сохранением состояния, DHCPv6

Суть измерения — разница между двумя строками с IPv6. На порту мониторинга рядом с выданным сервером адресом появился и адрес, составленный ядром из объявленного префикса, а на контрольном порту с опущенным флагом A такого адреса не появилось, хотя оба интерфейса всё время были активны и обслуживались одним устройством. Добавление одного ключевого слова, таким образом, — реальная и единственная причина наблюдаемого изменения. Тесты доступно-

сти к адресам, выданным сервером, и к адресу, составленному автоконфигурацией, прошли без потерь с временем отклика около одной миллисекунды.

3.3.2 Проверка аудита IPv6

Аудит проверен на перемычке между портами, где обе линии поднялись на скорости 1000 Мбит/с в полнодуплексном режиме. Разбор сообщения, захваченного на порту `snd0` после включения `dnsmasq`, показан на рисунке 3.10. Прибор правильно определил, что объявляющим маршрутизатором является его собственный второй порт, поскольку физический адрес отправителя совпадает с одним из его интерфейсов и сообщение принято по кабелю. Из сообщения он извлёк префикс `fd00:1::/64` с флагами L и A, время действительности 43 200 секунд и MTU 1500 байт. Нулевое время жизни маршрутизатора, введённое намеренно (подраздел 3.1.2), он оценил лишь как информацию, потому что сообщение исходит от собственного прибора. Он также отметил, что сообщение одновременно несёт флаг M и префикс с флагом A, то есть предлагает станциям и адрес от сервера, и автоконфигурацию; это точное описание поведения `dnsmasq` после добавления `slaac`, а не ошибка, и это соответствует таблице 3.3.

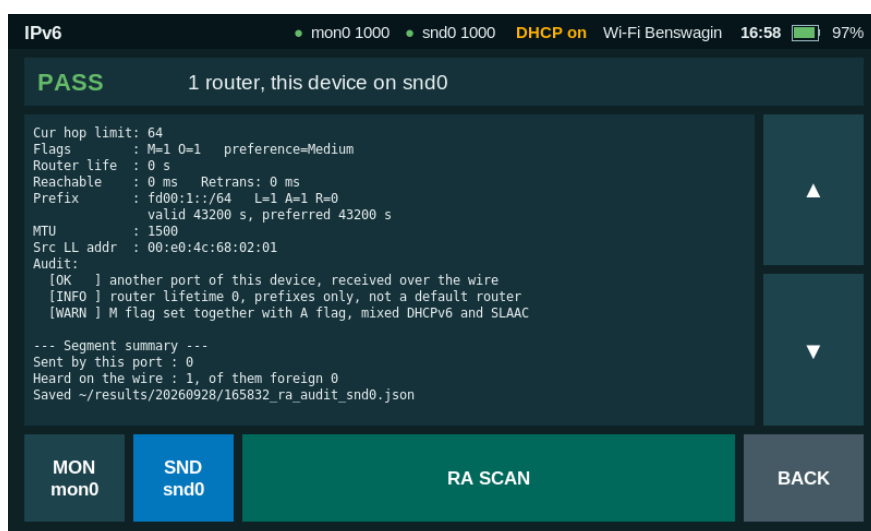


Рис. 3.10: Аудит сообщения Router Advertisement, принятого от второго порта прибора

Той же схемой удалось подтвердить таблицу 3.2 измерением самого прибора: с исходной настройкой аудит показал у префикса опущенный флаг A и предупреждение, что автоконфигурация запрещена, после добавления `slaac` — поднятый флаг. Сами проверки на реальном трафике проверить не удалось, потому что в тестируемом сегменте нет неправильно настроенного маршрутизатора. Поэтому они проверены на программно собранном стенде из двух временных пространств имён, соединённых виртуальной парой интерфейсов, куда было вброшено намеренно испорченное сообщение: Нор Limit 64, адрес источника вне локального канала, префикс длиной 48 бит с поднятым флагом A, предпочтительное время жизни больше времени действительности, MTU 1200 байт и другой физический адрес в опции. Прибор отметил все несоответствия сразу и оценил сообщение как сообщение чужого маршрутизатора.

Поиск соседей проверен на той же перемычке (рисунок 3.11). Прибор нашёл единственного соседа, вычислил из его физического адреса идентификатор интерфейса, установил совпадение с адресом локального канала и для обоих префиксов на порту составил адрес, который проверил как достижимый. Нагляднее всего, однако, сравнение обоих протоколов в один и тот же момент, сведённое в таблице 3.4. Сканирование ARP не нашло никого, потому что порты находятся в разных подсетях и на другом конце не работает клиент DHCP. IPv6 на том же кабеле нашёл того же соседа и проверил его адрес, без единого процесса настройки. Утверждение из подраздела 3.2.5

подкреплено, таким образом, измерением.

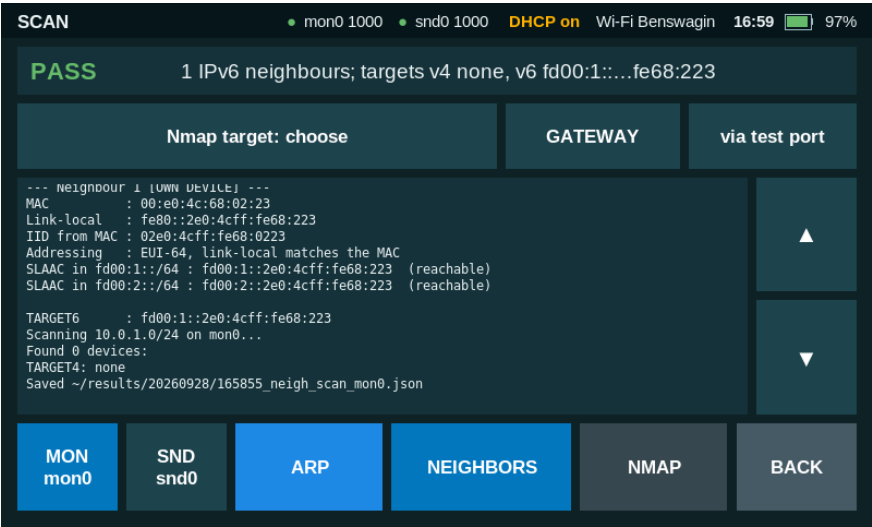


Рис. 3.11: Поиск соседей: адреса, вычисленные по EUI-64 и проверенные тестом доступности; сканирование ARP без результата

Таблица 3.4: Поиск другой стороны на одном кабеле обоими протоколами

Протокол	Способ	Результат
IPv4	сканирование ARP в диапазоне 10.0.1.0/24	станций не найдено
IPv6	запрос на ff02::1 и таблица соседей	сосед найден, адрес проверен

3.3.3 Точность и пределы анализатора

Точность анализатора проверена на перемычке между портами так: тот же процесс, что считал статистику, одновременно сохранял захваченный трафик в файл, который затем независимо пересчитан утилитой tcpdump с фильтрами по протоколам и утилитой capinfos. При смешанном трафике из ста ring по IPv4, ста ring по IPv6 и пятисекундной передачи TCP все счётчики совпали точно (таблица 3.5). При передаче UDP со скоростью 10 Мбит/с в течение тридцати секунд анализатор насчитал 37 503 датаграммы, а iperf3 отправил 37 501 датаграмму без потерь; разница в две датаграммы — это два служебных сообщения, которыми iperf3 обменивается в начале теста. Разбор отдельных кадров на экране TRAFFIC дополнительно сравнён с утилитой tshark, то есть с разбором анализатора Wireshark: на 58 кадрах всех поддерживаемых типов совпали все 781 сравниваемое поле.

Таблица 3.5: Число кадров по протоколам: анализатор прибора и независимый пересчёт сохранённого файла

Протокол	Анализатор прибора	Пересчёт tcpdump
TCP	11 251	11 251
UDP	0	0
ICMP	200	200
ICMPv6	202	202
ARP	4	4
всего	11 657	11 657

Предел анализатора искали, постепенно повышая скорость передачи UDP. Выразить его в ме-

габитах в секунду нельзя, потому что решает число кадров: обработка каждого кадра стоит одинаковой работы независимо от размера. С короткими кадрами размером 162 байта анализатор без потерь захватил 22 498 кадров в секунду; при 26 247 кадрах в секунду потерял уже 11,8 %, а выше установился примерно на 23 000 кадрах в секунду. Узкое место — одно ядро процессора, на котором идёт разбор заголовков и которое было загружено на 99 %, тогда как у остальных ядер был запас. Для кадров полной длины 1514 байт этот предел соответствует примерно 270 Мбит/с, то есть примерно потолку шины USB 2.0: длинные кадры анализатор успевает, короткие на высокой скорости — нет.

Эффект фильтра в ядре измерен при передаче TCP со скоростью 50 Мбит/с с добавленными сообщениями ICMPv6. Без фильтра разбор загружал процессор на 27,2 %, с фильтром `icmp6` — всего на 0,4 %, и статистика содержала только сообщения ICMPv6. Кадры, не прошедшие фильтр, до программы действительно не доходили.

3.3.4 Производительность генератора пакетов

Генератор отправил тысячу сообщений ICMPv6 Echo Request со скоростью сто сообщений в секунду с порта `snd0`, и анализатор на порту `mon0` насчитал ровно тысячу (рисунок 3.6); тот же результат дали все шесть поддерживаемых типов пакетов и обратное направление. Наибольшая скорость измерялась без захвата, по счётчику принятых кадров адаптера. С кадрами размером 100 байт генератор достиг 83 764 кадров в секунду при загрузке одного ядра на 97 %, с кадрами 1514 байт передал 157 Мбит/с при загрузке 17 %, во всех случаях без потерь на приёмной стороне. Короткие кадры, таким образом, ограничивает процессор, длинные — шина. Смысл генератора при этом не в максимальной скорости: `iperf3` с короткими датаграммами отправляет больше кадров в секунду, как показывает подраздел 3.3.5. Генератор служит для отправки точно заданного числа пакетов определённого типа и проверки, что все они дошли.

3.3.5 Измерение пропускной способности

Пропускная способность по TCP измерялась против ноутбука с обоих портов и обеими версиями IP. Результаты сведены в таблице 3.6, один из них показан на рисунке 3.12. Все четыре измерения лежат между 273 и 276 Мбит/с и совпадают с измерением, проведённым ранее в лаборатории, где получилось 276 Мбит/с для IPv4 и 274 Мбит/с для IPv6. Разница между протоколами — единицы мегабит и соответствует более длинному заголовку IPv6, разобранным в разделе 2.1.

Таблица 3.6: Пропускная способность по TCP, тест длительностью десять секунд

Схема	Протокол	Пропускная способность [Мбит/с]
порт <code>mon0</code> против ноутбука	IPv4	274,0
порт <code>mon0</code> против ноутбука	IPv6	274,2
порт <code>snd0</code> против ноутбука	IPv4	273,3
порт <code>snd0</code> против ноутбука	IPv6	275,0
перемычка между портами	IPv6	150,5

Значения лежат чуть ниже границы 300 Мбит/с, которую производитель указывает для Ethernet, подключённого через шину USB 2.0 [23]. Измерение, таким образом, определило не пропускную способность линии, а потолок самого анализатора. На перемычке между портами потолок ещё примерно вдвое ниже, около 150 Мбит/с: каждый кадр проходит одну и ту же шину дважды — один раз при отправке одним адаптером и второй раз при приёме другим. Результаты с перемычки и с измерения против чужой станции поэтому смешивать нельзя. Прибор пригоден, чтобы проверить работоспособность соединения, сравнить протоколы и выявить грубые неисправности, но не для сертификации гигабитной линии на номинальной скорости.

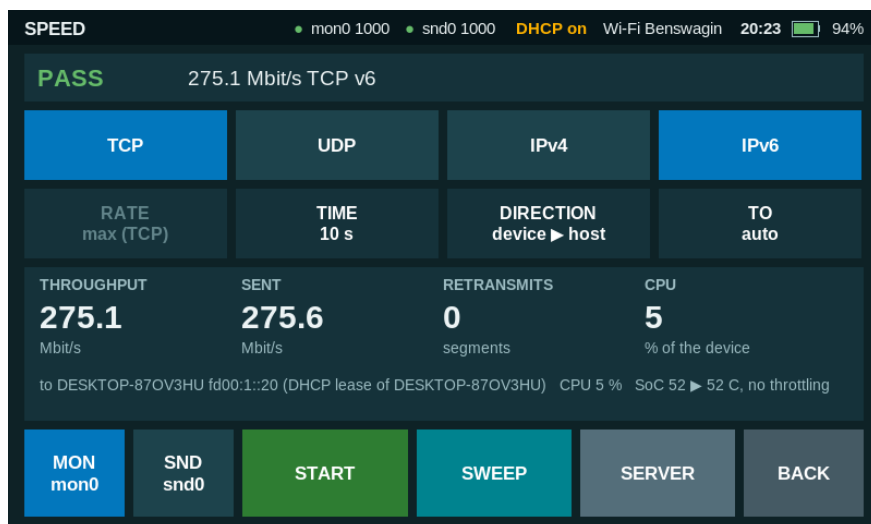


Рис. 3.12: Измерение пропускной способности по TCP и IPv6 против ноутбука

Измерение по UDP со скоростью 100 Мбит/с прошло и против ноутбука в течение десяти секунд, и на переключке в течение тридцати секунд без единой потерянной датаграммы, с джиттером 0,17 мс. При заданной скорости 400 Мбит/с, то есть выше возможностей платформы, прибор показал реально достигнутую скорость 265 Мбит/с, а не заданное значение.

Результаты развёртки по размерам кадра сведены в таблице 3.7 и на рисунке 3.13. Каждый размер измерялся десять секунд с отправкой настолько быстро, насколько мог прибор, и вся развёртка для каждого протокола заняла около 80 секунд (рисунок 3.14). Ход кривых показывает оба предела, о которых говорит теория. На коротких кадрах прибор отправляет до 147 тысяч кадров в секунду, но пропускная способность низкая, потому что процессор отправляющего инструмента загружен полностью. С ростом размера кадра число кадров падает, а пропускная способность по кадрам приближается к 276 Мбит/с, то есть к потолку шины. Потери не более 1,4 % появлялись только на некоторых коротких кадрах и менялись от измерения к измерению, потому что возникали на приёмной стороне.

Таблица 3.7: Развёртка по размерам кадра по UDP, анализатор против ноутбука

Кадр [Б]	IPv4		Кадр [Б]	IPv6	
	Мбит/с	тыс. кадров/с		Мбит/с	тыс. кадров/с
64	67,1	131,1	82	81,2	123,8
128	150,5	147,0	128	137,1	133,9
256	224,6	109,7	256	224,0	109,4
512	247,4	60,4	512	246,8	60,3
1024	268,5	32,8	1024	269,8	32,9
1280	273,6	26,7	1280	272,1	26,6
1518	276,4	22,8	1518	276,5	22,8

Режим сервера проверен так: измерение против прибора запускал ноутбук (рисунок 3.15). Прибор принял 249 Мбит/с по TCP, а в обратном направлении отправил 272 Мбит/с. После нажатия STOP в пространстве имён порта не осталось ни одного процесса сервера.

3.3.6 Подключение обычного компьютера

Ноутбук с Windows 11 получил от прибора на обоих адаптерах адреса 10.0.1.20 и 10.0.2.20, а для IPv6 — по три адреса: выданный DHCPv6, составленный автоконфигурацией с идентификатором,

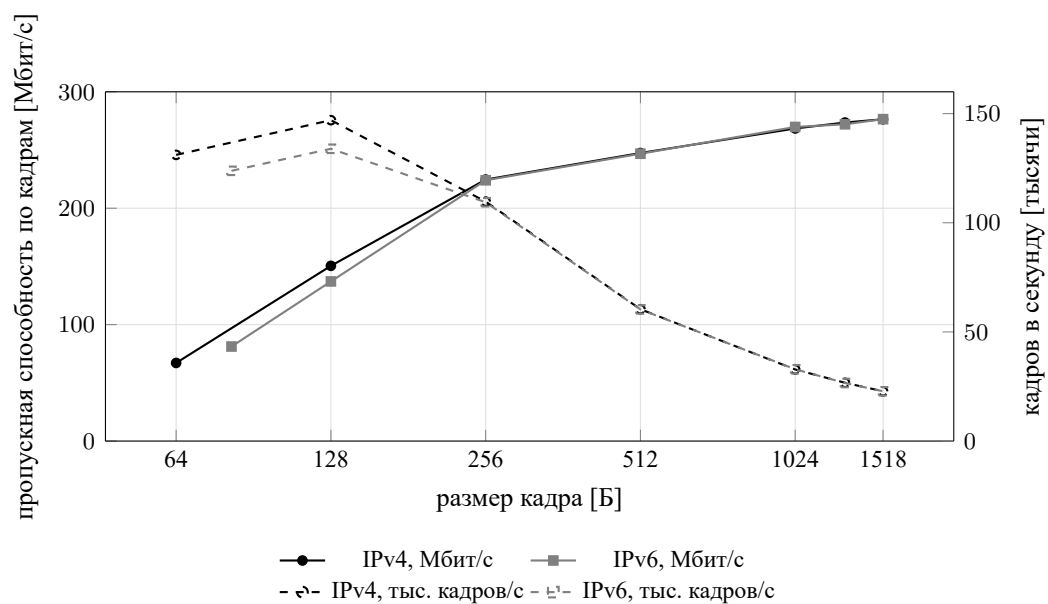


Рис. 3.13: Пропускная способность по кадрам и число кадров в секунду в зависимости от размера кадра

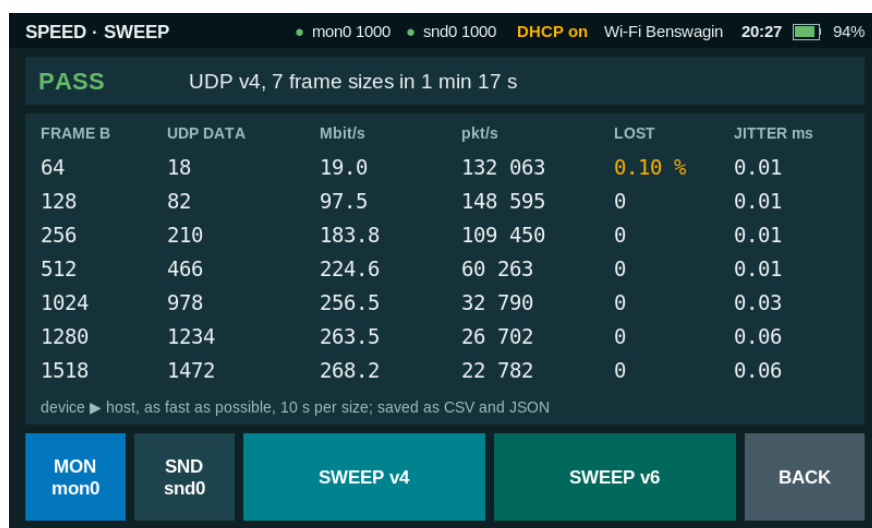


Рис. 3.14: Результат развёртки по размерам кадра на дисплее прибора

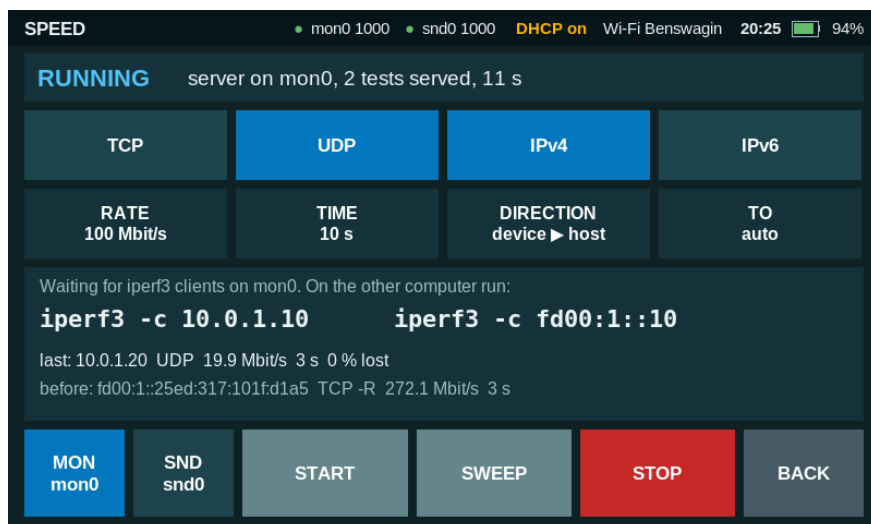


Рис. 3.15: Режим сервера: ноутбук измеряет пропускную способность против анализатора

не выведенным из физического адреса, и вдобавок временный адрес. Маршрута по умолчанию через анализатор не появилось, и ноутбук остался подключён к Интернету через Wi-Fi, что подтвердило правильность решения из подраздела 3.1.2.

Первый ping остался без ответа. Windows считает неизвестную сеть общественной и блокирует на ней входящие сообщения Echo Request; на сообщения, отправленные на групповой адрес всех узлов, не отвечает вовсе. При этом прибор правильно сообщил, что станция подключена, поскольку она ответила на запрос ARP и Neighbor Solicitation, и что ping блокирует её межсетевой экран. После разрешения входящих Echo Request на обоих тестовых адаптерах тесты во всех четырёх сочетаниях порта и протокола прошли без потерь с временем отклика от 1,2 до 1,8 мс.

Измерение по UDP поверх IPv6 поначалу не удавалось установить. Журнал сервера iperf3 на ноутбуке показал, что сервер отвечает со своего временного адреса, хотя анализатор посылал запрос на адрес, выданный DHCPv6. Временные адреса служат защите приватности, и система предпочитает их при отправке [7]; не привязанный к адресу сервер iperf3 поэтому ответил не с того адреса, к которому к нему обратились, и анализатор такой ответ законно отбросил. После запуска сервера с привязкой к конкретному адресу измерение прошло без ошибок. Теперь в такой ситуации прибор показывает объяснение и рекомендацию.

Самым важным выводом стало поведение передачи UDP в обратном направлении, то есть с ноутбука в анализатор. При 20 Мбит/с потерь почти не было, при 50 Мбит/с они составили 24 %, при 100 Мбит/с — 57 %, хотя в том же направлении TCP передал 250 Мбит/с. Буфер сокета на анализаторе не переполнялся, а ноутбук по своим счётчикам отправил все датаграммы. Решил дело счётчик пропущенных кадров адаптера: за одно измерение он вырос на 13 559, а iperf3 насчитал 13 543 потерянные датаграммы. Кадры, таким образом, выбрасывает сам адаптер анализатора. iperf3 в Windows выдерживает заданную среднюю скорость, но отправляет датаграммы пачками на полной скорости гигабитной линии, и адаптер на шине USB 2.0 не успевает передать их дальше. TCP приспособляется к этому управлением потоком, UDP — нет. Это реальный предел прибора как приёмника: пачечный трафик со скоростью выше пропускной способности шины теряется уже на входе. Поэтому прибор теперь записывает значение этого счётчика к каждому измерению и объясняет, где произошла потеря.

3.3.7 Питание и время автономной работы

Время автономной работы измерено разрядным тестом в типичном режиме, то есть с пассивным прослушиванием на порту мониторинга, включённым дисплеем и подключённой сетью Wi-Fi.

Прибор проработал от полностью заряженного аккумулятора 6 часов 43 минуты, пока напряжение ячеек не упало ниже 9,6 В и служба штатно его не выключила (рисунок 3.16). Аккумулятор за это время отдал 2,51 А·ч и 28,2 Вт·ч, то есть 84 % номинальной ёмкости ячеек, при среднем потреблении 4,2 Вт. Напряжение падало почти равномерно, кроме последних получаса, когда оно падало вдвое быстрее. Худший случай из таблицы 3.1 отдельно не измерялся; из той же энергии и большего потребления получается оценка около четырёх часов.

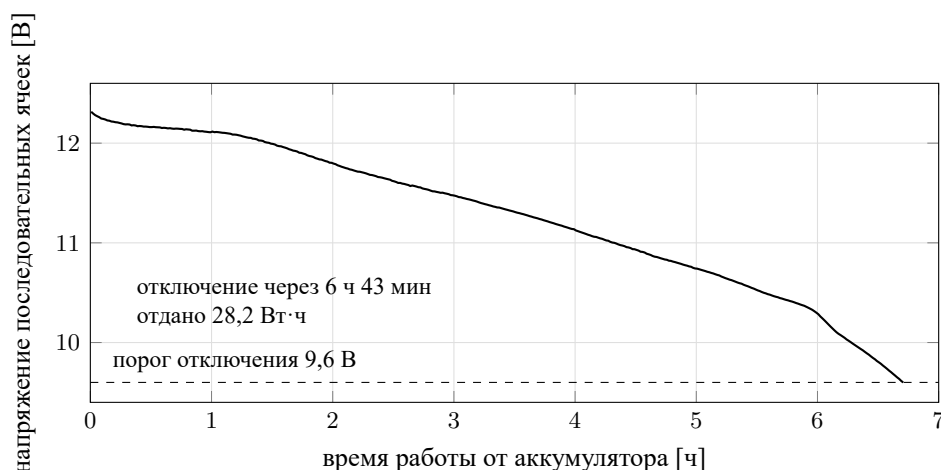


Рис. 3.16: Напряжение аккумулятора во время разрядного теста в типичном режиме

По той же записи построена кривая, по которой прибор переводит напряжение в оставшийся заряд в процентах и оценивает оставшееся время работы. Отдельно измерено, что даёт блокировка экрана: выключение выхода HDMI снизило потребление от аккумулятора с 4,53 до 4,18 Вт, то есть на 0,35 Вт, что продлит работу заблокированного прибора примерно на полчаса. Выключение линий тестовых портов потребление не снизило вовсе.

3.3.8 Сканирование сети и проверка изоляции пространств имён

Сканер Nmap запускается с экрана SCAN — либо через тестовый порт внутри его пространства имён, либо через интерфейс Wi-Fi с автоматической подстановкой шлюза по умолчанию беспроводной сети. На рисунке 3.17 показано сканирование второго порта того же прибора через перемычку. Сканер нашёл станцию, определил производителя её адаптера по физическому адресу и установил, что из ста самых распространённых портов не открыт ни один; прибор, таким образом, не выставляет на своих тестовых портах никаких служб.

Проверка сканера выявила и дефект первой версии приложения, который одновременно служит доказательством изоляции. В режиме проводного сканирования сканер жёстко запускался в пространстве имён `analyzer_sender`, поэтому попытка сканировать адрес `10.0.1.20` из подсети порта мониторинга заканчивалась сообщением `failed to determine route`. Тот же адрес из пространства `analyzer_monitor` был доступен без проблем. Это была не неисправность сети, а следствие того, что между пространствами намеренно нет пути: трафик из одного пространства не попадёт в другое, даже если об этом прямо попросить. Инструменты теперь всегда запускаются в пространстве имён порта, выбранного на экране.

3.3.9 Дефекты, выявленные при повторной проверке

Повторная проверка выявила несколько дефектов, которые не проявились при первом проходе измерений и могли остаться скрытыми до применения прибора вне лаборатории.

Самый серьёзный касался исходного счётчика трафика. Он различал протоколы по словам в

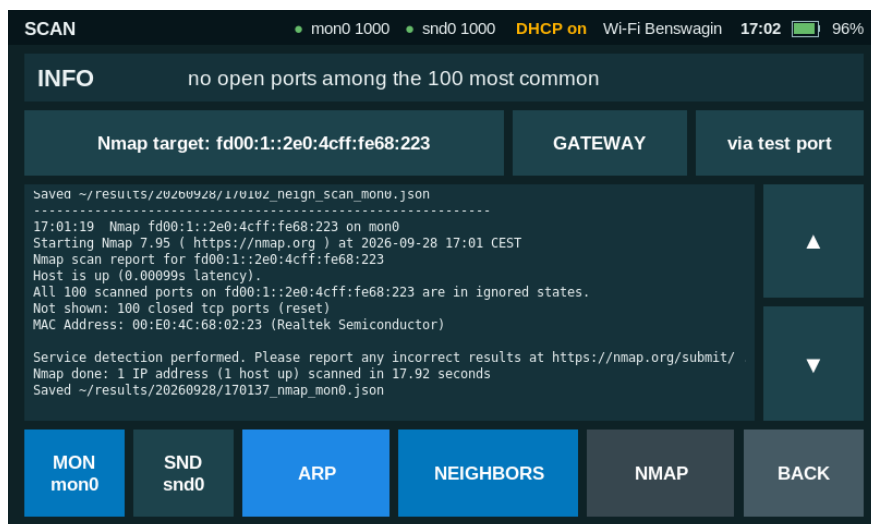


Рис. 3.17: Сканирование Nmap через тестовый порт

текстовом выводе `tcpdump`, но строки с сегментами TCP слова «TCP» не содержат. При контрольном измерении счётчик насчитал ноль сегментов TCP, тогда как одновременно сохранённый файл содержал их 1202; счётчики UDP и ICMP при этом были верны. Дефект устранён заменой счётчика анализатором из подраздела 3.2.2, который читает двоичные заголовки.

Скрипт инициализации при повторном запуске в том же сеансе системы давал сбой, потому что пытался перенести интерфейс в новое пространство имён раньше, чем ядро после удаления старого пространства вернуло его в основное. Дефект исчез с переходом на службу, которая пространства заново не создаёт, а только достраивает недостающее.

Ещё один дефект управляющего приложения был обнаружен способом, заслуживающим упоминания. На снимке экрана лента выглядела прерывистой, часть текста дублировалась, хотя содержимое ленты, прочитанное программно, было безупречным. Причина была в том, что отдельные задачи писали в элементы интерфейса прямо из своих потоков, чего использованная библиотека не допускает и что в крайнем случае ведёт к падению приложения. Запись в элементы интерфейса теперь откладывается в главный поток.

Два меньших дефекта выявило только подключение реального компьютера и съёмка экранов для этой работы. Записи об адресах, выданных DHCP, живут двенадцать часов, и прибор после отключения ноутбука пытался обратиться к станции, которой уже не было; кандидаты в цели теперь проверяются заново. Сканер Nmap, не успевший исследовать станцию за заданный лимит времени, оценивался как сканирование без открытых портов; теперь такое сканирование оценивается как незавершённое. Общий урок: то, что на дисплее, нужно сверять с независимо полученными данными, а не полагаться на то, что видно.

Глава 4

Заключение

Цель этой бакалаврской работы — спроектировать и реализовать переносное устройство для тестирования и мониторинга компьютерных сетей, которое содержит генератор и анализатор трафика протоколов IPv4 и IPv6 и позволяет измерять такие параметры сети, как задержка, пропускная способность и потери. Эта цель достигнута.

В теоретической части описана проблематика генерирования сетевого трафика, включая разницу между измерением в битах и в кадрах в секунду, рекомендуемые размеры кадра и расчёт джиттера, и проблематика мониторинга сетей, включая фильтрацию трафика в ядре, сводные статистики и пределы самого захвата. Кроме того, разобраны адресация и автоконфигурация протокола IPv6 и технология пространств имён, на которой построен прибор.

В практической части создан прототип на основе одноплатного компьютера Raspberry Pi с сенсорным дисплеем, двумя внешними сетевыми адаптерами и аккумуляторным модулем. После включения прибор сразу запускает управляющее приложение и в типичном режиме работает от аккумулятора 6 часов 43 минуты. Он содержит анализатор трафика со сводными статистиками, разбором отдельных кадров, фильтрами в ядре и сохранением в формат pcap, генератор пакетов шести типов, измерение задержки тестом ping и измерение пропускной способности, потерь и джиттера инструментом iperf3, включая развёртку по размерам кадра и режим сервера. Для IPv6 он предлагает аудит сообщений Router Advertisement и поиск соседей с вычислением их адресов.

Решение проверено измерениями — и на перемычке между двумя портами прибора, и против чужой станции под Linux, и против ноутбука под Windows. Анализатор насчитал столько же кадров, сколько независимый пересчёт сохранённого файла, и его разбор совпал с анализатором Wireshark. Генератор доставил все отправленные пакеты во второе пространство имён. Пропускная способность по TCP против ноутбука составила 273–276 Мбит/с для обоих протоколов IP.

Собственный вклад автора

Первый вклад — архитектура прибора, в которой оба тестовых интерфейса логически разделены пространствами имён ядра Linux, а каждая измерительная функция — отдельный скрипт, который работает в пространстве выбранного порта и передаёт результат управляющему приложению в машиночитаемом виде. Разделение устраняет конфликты маршрутизации между двумя интерфейсами в одном сегменте и проверено измерением: попытка обратиться к цели из неправильного пространства заканчивается недоступностью маршрута, а пакеты генератора попадают во второе пространство только по кабелю. Каждый результат сохраняется вместе со временем и контрольной суммой использованного скрипта, так что его происхождение можно проследить.

Второй вклад — разбор автоконфигурации адресов IPv6 на стороне анализатора. Показано, что dnsmasq при диапазоне, заданном одним адресом, опускает у объявленного префикса флаг A, и что после добавления ключевого слова slaac ядро станции само составляет адрес без какого-

либо процесса в пространстве пользователя; контрольный порт с исходной настройкой подтвердил, что другая причина на результат не влияла. На этот разбор опирается модуль аудита, который разбирает сообщение Router Advertisement, оценивает его по требованиям стандартов и различает отправку собственного порта, второго порта того же прибора и чужого маршрутизатора, а также поиск соседей, который на одном и том же кабеле показал, что IPv6 находит другую сторону там, где IPv4 не находит ничего.

Третий вклад — пара генератора и анализатора трафика с проверенной точностью и измеренными пределами. Анализатор без потерь захватывает около 22 500 кадров в секунду, генератор отправляет до 84 000 кадров в секунду, и оба предела задаются производительностью процессора, а не объёмом данных. Измерения показали также, что на переключке между портами пропускная способность вдвое ниже, потому что каждый кадр проходит шину USB дважды, и что адаптер прибора выбрасывает кадры UDP, если они приходят пачками быстрее, чем шина успевает их передать. Эти выводы определяют, как толковать измеренные числа.

Четвёртый вклад — воплощение устройства как самостоятельного прибора. Он работает от собственного аккумулятора с измерением заряда и штатным выключением при разряде, управляется пальцем на резистивном дисплее с элементами управления, рассчитанными на касание, и после подключения к чужой сети сам ничего не отправляет. Подключённой станции он выдаёт только адрес, а не шлюз по умолчанию, поэтому не нарушает её существующее подключение к Интернету. Для не прошедших тестов он объясняет оператору причину и советует, что делать дальше.

Ограничения и рекомендации по дальнейшему развитию

Границы прибора нужно очертить честно. Ethernet использованной платформы подключён через шину USB 2.0, поэтому прибор достигает не более примерно 276 Мбит/с; он пригоден, чтобы проверить работоспособность соединения, сравнить протоколы в одинаковых условиях и выявить грубые неисправности, но не для сертификации гигабитной линии. Устранение этого ограничения потребовало бы перехода на более новое поколение одноплатного компьютера с контроллером Ethernet не на шине USB. Это же касается приёма пачечного трафика UDP и предела анализатора в кадрах в секунду.

Для дальнейшего развития есть несколько направлений. Ближайшее — корпус, в котором все компоненты будут жёстко закреплены; прибор уже работает от аккумулятора, но его части пока соединены свободно. Второе направление — автоматический тест розетки одной кнопкой, который после подключения к сети проверил бы линию, доступность сервера DHCP и маршрутизатора IPv6, шлюз по умолчанию и разрешение имён, как это делают коммерческие тестеры. Далее — определение свойств порта, то есть согласованной скорости, данных протоколов обнаружения соседних устройств и меток VLAN, затем определение наибольшего размера пакета, проходящего по пути, экспорт результатов на носитель и уточнение сенсорного управления калибровкой резистивной панели. Модуль аудита IPv6 подсказывает и направление безопасности: проверки, отработанные в этой работе на намеренно испорченном сообщении, в рабочей сети смогли бы обнаружить неправильно настроенный или несанкционированно подключённый маршрутизатор.

Литература

- [1] R. A. Nastase. *IPv6 Protocol for Beginners: Your Quick Guide for Learning the Fundamentals of the IPv6*. Blurb, 2024. ISBN: 979-8210198082.
- [2] S. Deering и R. Hinden. *Internet Protocol, Version 6 (IPv6) Specification*. Request for Comments 8200. RFC Editor, июль 2017. DOI: 10.17487/RFC8200. URL: <https://www.rfc-editor.org/info/rfc8200> (дата обр. 03.08.2026).
- [3] Apaula Aotearoa Tuifaiga, Priyanka Ashreena Ram и S. S. Kolahi. «Performance Comparison of IPv6 in 802.11ac WLAN in Windows and Linux Environment». В: *TENCON 2021 – 2021 IEEE Region 10 Conference (TENCON)*. 2021, с. 799—804. DOI: 10.1109/tencon54134.2021.9707449.
- [4] R. Hinden и S. Deering. *IP Version 6 Addressing Architecture*. Request for Comments 4291. RFC Editor, февр. 2006. DOI: 10.17487/RFC4291. URL: <https://www.rfc-editor.org/info/rfc4291> (дата обр. 03.08.2026).
- [5] R. Hinden и B. Haberman. *Unique Local IPv6 Unicast Addresses*. Request for Comments 4193. RFC Editor, окт. 2005. DOI: 10.17487/RFC4193. URL: <https://www.rfc-editor.org/info/rfc4193> (дата обр. 03.08.2026).
- [6] F. Gont. *A Method for Generating Semantically Opaque Interface Identifiers with IPv6 Stateless Address Autoconfiguration (SLAAC)*. Request for Comments 7217. RFC Editor, апр. 2014. DOI: 10.17487/RFC7217. URL: <https://www.rfc-editor.org/info/rfc7217> (дата обр. 04.08.2026).
- [7] F. Gont и др. *Temporary Address Extensions for Stateless Address Autoconfiguration in IPv6*. Request for Comments 8981. RFC Editor, февр. 2021. DOI: 10.17487/RFC8981. URL: <https://www.rfc-editor.org/info/rfc8981> (дата обр. 28.09.2026).
- [8] T. Narten и др. *Neighbor Discovery for IP version 6 (IPv6)*. Request for Comments 4861. RFC Editor, сент. 2007. DOI: 10.17487/RFC4861. URL: <https://www.rfc-editor.org/info/rfc4861> (дата обр. 03.08.2026).
- [9] S. Thomson, T. Narten и T. Jinmei. *IPv6 Stateless Address Autoconfiguration*. Request for Comments 4862. RFC Editor, сент. 2007. DOI: 10.17487/RFC4862. URL: <https://www.rfc-editor.org/info/rfc4862> (дата обр. 03.08.2026).
- [10] T. Mrugalski и др. *Dynamic Host Configuration Protocol for IPv6 (DHCPv6)*. Request for Comments 8415. RFC Editor, нояб. 2018. DOI: 10.17487/RFC8415. URL: <https://www.rfc-editor.org/info/rfc8415> (дата обр. 03.08.2026).
- [11] ESnet. *iperf3: A TCP, UDP, and SCTP network bandwidth measurement tool*. 2026. URL: <https://github.com/esnet/iperf> (дата обр. 28.09.2026).
- [12] Zoltán Gál и др. «Performance evaluation of massively parallel and high speed connectionless vs. connection oriented communication sessions». В: *Advances in Engineering Software* 157-158 (2021), с. 103010. DOI: 10.1016/j.advengsoft.2021.103010.
- [13] T. Carter. *Network Security with Python and Scapy: A Beginner's Guide to Packet Analysis, Network Monitoring, and Security Automation*. Kindle Edition, 2024.

- [14] S. Bradner и J. McQuaid. *Benchmarking Methodology for Network Interconnect Devices*. Request for Comments 2544. RFC Editor, март 1999. DOI: 10.17487/RFC2544. URL: <https://www.rfc-editor.org/info/rfc2544> (дата обр. 28.09.2026).
- [15] C. Popoviciu и др. *IPv6 Benchmarking Methodology for Network Interconnect Devices*. Request for Comments 5180. RFC Editor, май 2008. DOI: 10.17487/RFC5180. URL: <https://www.rfc-editor.org/info/rfc5180> (дата обр. 28.09.2026).
- [16] H. Schulzrinne и др. *RTP: A Transport Protocol for Real-Time Applications*. Request for Comments 3550. RFC Editor, июль 2003. DOI: 10.17487/RFC3550. URL: <https://www.rfc-editor.org/info/rfc3550> (дата обр. 28.09.2026).
- [17] S. Bradner и др. *Applicability Statement for RFC 2544: Use on Production Networks Considered Harmful*. Request for Comments 6815. RFC Editor, нояб. 2012. DOI: 10.17487/RFC6815. URL: <https://www.rfc-editor.org/info/rfc6815> (дата обр. 28.09.2026).
- [18] Steven McCanne и Van Jacobson. «The BSD Packet Filter: A New Architecture for User-level Packet Capture». В: *Proceedings of the USENIX Winter 1993 Conference*. San Diego: USENIX Association, 1993. URL: <https://www.tcpdump.org/papers/bpf-usenix93.pdf> (дата обр. 28.09.2026).
- [19] Wireshark Foundation. *Wireshark User's Guide: Chapter 8. Statistics*. 2026. URL: https://www.wireshark.org/docs/wsug_html_chunked/ChStatistics.html (дата обр. 28.09.2026).
- [20] Mustafa Coşar и Harun Emre Kiran. «Measurement of Raspberry Pi performance in network traffic analysis». В: *2018 26th Signal Processing and Communications Applications Conference (SIU)*. 2018, с. 1—4. DOI: 10.1109/siu.2018.8404395.
- [21] Shyava Tripathi и Rishi Kumar. «Raspberry Pi as an Intrusion Detection System, a Honeypot and a Packet Analyzer». В: *2018 International Conference on Computational Techniques, Electronics and Mechanical Systems (CTEMS)*. 2018, с. 80—85. DOI: 10.1109/ctems.2018.8769135.
- [22] Febrian Rachmad Hariawan и Septia Ulfa Sunaringtyas. «Design an Intrusion Detection System, Multiple Honeypot and Packet Analyzer Using Raspberry Pi 4 for Home Network». В: *2021 17th International Conference on Quality in Research (QIR)*. 2021, с. 43—48. DOI: 10.1109/qir54354.2021.9716189.
- [23] Raspberry Pi Ltd. *Raspberry Pi 3 Model B+ Product Brief*. 2025. URL: <https://datasheets.raspberrypi.com/rpi3/raspberry-pi-3-b-plus-product-brief.pdf> (дата обр. 03.08.2026).
- [24] Waveshare. *UPS Module 3S*. 2026. URL: https://www.waveshare.com/wiki/UPS_Module_3S (дата обр. 28.09.2026).
- [25] S. Alexander и R. Droms. *DHCP Options and BOOTP Vendor Extensions*. Request for Comments 2132. RFC Editor, март 1997. DOI: 10.17487/RFC2132. URL: <https://www.rfc-editor.org/info/rfc2132> (дата обр. 28.09.2026).

Приложение А

Исходные коды измерительных скриптов

В оригинале работы (приложение А) напечатаны исходные коды семи ключевых скриптов: `setup_network.sh`, `sniff_stats.py`, `pkt_gen.py`, `perf_test.py`, `ra_audit.py`, `neigh_scan.py` и `arp_scan.py`. В этом переводе коды не повторяются: они одинаковы в обеих версиях и лежат в папке `SourceCodes/` проекта.

Приложение В

Обзор программного обеспечения прибора

Программное обеспечение хранится в каталоге `~/diploma_project/` на карте памяти прибора; системные файлы лежат в обычных местах системы. Полные исходные коды всех файлов входят в электронное приложение к работе. Обзор приведён в таблице В.1; скрипты, напечатанные в приложении А оригинала, отмечены звёздочкой.

Таблица В.1: Файлы программного обеспечения прибора

Файл	Назначение
<i>Измерительные скрипты, запускаются в пространстве имён порта</i>	
<code>setup_network.sh*</code>	подготовка портов, пространств имён и сервера dnsmasq
<code>sniff_stats.py*</code>	анализатор трафика, статистика, сохранение в pcap и CSV
<code>pktdecode.py</code>	разбор заголовков кадра без Scapy, для статистики и для разбора по слоям
<code>pkt_gen.py*</code>	генератор пакетов шести типов
<code>perf_test.py*</code>	измерения iperf3 и развёртка по размерам кадра
<code>perf_server.py</code>	сервер iperf3 на выбранном порту
<code>path_test.py</code>	тест доступности и задержки утилитой ping
<code>targets.py</code>	общий поиск цели и объяснение, почему цели нет
<code>ra_audit.py*</code>	аудит сообщений Router Advertisement
<code>neigh_scan.py*</code>	поиск соседей IPv6
<code>arp_scan.py*</code>	сканирование ARP
<i>Управляющее приложение</i>	
<code>gui_app.py</code>	главный экран, запуск задач, экран приветствия и блокировки
<code>ui_kit.py</code>	общий вид экранов, строка состояния, кнопки
<code>screens.py</code>	экраны PATH, SCAN, IPv6, DHCP / RA и SYSTEM
<code>traffic.py</code>	экран TRAFFIC с разбором кадра и фильтрами
<code>generator.py</code>	экран GENERATOR
<code>speed.py</code>	экран SPEED с развёрткой и режимом сервера
<code>results.py</code>	сохранение результатов в файлы JSON
<code>wifi_dialog.py</code>	подключение к сети Wi-Fi и клавиатура на экране
<i>Системные службы</i>	
<code>port_status.py</code>	слежение за состоянием портов и запись в файл состояния
<code>ups_monitor.py</code>	измерение состояния аккумулятора и штатное выключение при разряде
<code>analyzer-port@.service</code>	подготовка порта при подключении адаптера
<code>10-analyzer-*.link</code>	постоянные имена портов по физическому адресу
<code>~/xinitrc</code>	запуск управляющего приложения вместо рабочего стола